# 07 搜索与索引

- 搜索与索引
- 基于顺序表的搜索
- 二叉搜索树

# 07 搜索与索引

- **搜索与索引**
- 基于顺序表的搜索
- 二叉搜索树

# 搜索(Search)

- **搜索是数据处理中最常见的一种操作**
  - **从一个数据对象的集合中寻找关键码或某个属性为给定值k的数据对象。如果找到则返回该数据对象，否则返回失败标志**

- **搜索操作涉及的数据结构有如下两种分类方式**
  - **静态和动态**
    - **静**态数据结构不考虑(**假**设不发生)**数据插入和**删除，只考虑搜索的算法及其效率
    - 动态数据结构要同时考虑数据插入、删除和搜索的算法及其效率
  - **内存和外存**
    - **基于内存的数据**结构是指整个数据结构完全放在内存中
    - **基于外存的数据**结构是指数据部分或全部存放在磁盘上，搜索过程需要将磁盘数据读入内存，为此需要充分考虑磁盘的读写特点

# 索引(Index)

- **索引**是用于支持搜索操作的数据结构及其操作，是提升大规模数据**搜索性能**的最有效的手段之一

- 设计一个索引结构需要从如下方面进行考虑
  - 能够支持对大规模数据的管理
  - 能够支持高效的搜索操作
  - 动态索引结构能够有效地支持频繁的插入、删除和修改操作
  - 需要考虑数据存储介质的特点，如以磁盘为存储介质的索引结构应考虑磁盘按块存取的特点

# 搜索算法衡量标准

- 在搜索过程中关键码的平均比较次数或平均读写磁盘次数(只适合于外部搜索)，称为**平均搜索长度**ASL(Average Search Length)，通常它是搜索结构中数据对象总数n或文件结构中物理块总数n的函数

- 另外，衡量一个搜索算法还要考虑算法所需要的存储量和算法的复杂性等问题

# 07 搜索与索引

- 搜索与索引

- 基于顺序表的搜索

- 二叉搜索树

# 基于顺序表的搜索

- 数据在顺序表中可以有两种存储方式
  - 无序的顺序表只能用顺序搜索的方式进行搜索
  - 有序的顺序表可以用折半、插值、斐波那契搜索等方式进行搜索

# 无序顺序表上的顺序搜索

- 顺序搜索：从表的第一个数据对象开始，顺序用各数据对象的关键码与给定值x进行比较，直到找到与其值相等的数据对象，则搜索成功，给出该数据对象在表中的位置；若整个表都已搜索完但未找到关键码与x相等的数据对象，则搜索失败，给出失败信息

- 顺序搜索的时间复杂性为O(n)，n为顺序表的长度

# 顺序搜索算法

```
// 顺序搜索关键码为x的对象, 第CurrentSize个
// 位置作为控制搜索自动结束的“监视哨”使用
template <class Type> int SearchList<Type> ::
Search ( const Type& x ) const {
    Element[CurrentSize].key = x;  // 监视哨
    int i = 0;
    // 从前向后顺序搜索
    while ( Element[i].key != x ) i++; //省去每次循环时的位置判断
    return i;
}
```

# 顺序搜索的平均搜索长度

- 设搜索第i个元素的概率为$p_i$，搜索到第i个元素所需比较次数为$c_i$，则搜索成功的平均搜索长度ASL = $\sum_{i=0}^{n-1} p_i * ci = \sum_{i=0}^{n-1} p_i * (i+1)$
- 若每个数据对象的搜索概率相等，即$p_i=1/n$，则搜索成功的平均搜索长度ASL = $\frac{1}{n} * \sum_{i=0}^{n-1}(i+1) = \frac{n+1}{2}$
- 搜索不成功时，则比较次数为n+1(设置监视哨的情形)

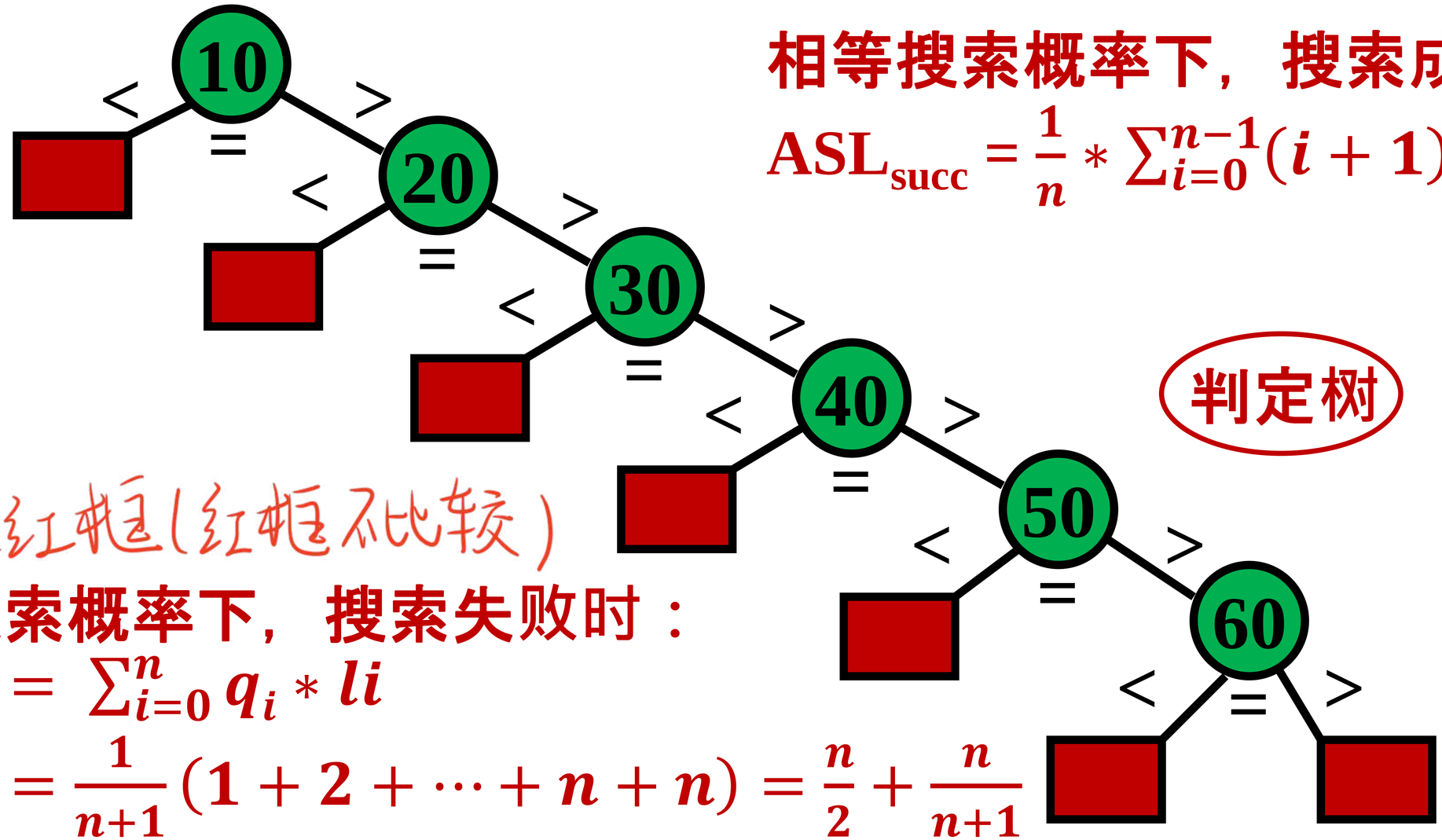
# 有序顺序表上的顺序搜索

- **在搜索不成功时不需要遍历整个顺序表**

```
// 顺序搜索关键码为x的数据对象
template <class Type> int SearchList<Type> ::
Search ( const Type& x ) const {
    for ( int i = 0; i < CurrentSize; i++ )
        if ( Element[i].key == x ) return i;  // 顺序搜索成功
        else if ( Element[i].key > x ) break;
    return −1;  // 顺序搜索失败
}
```

# 顺序搜索的平均搜索长度

**(10, 20, 30, 40, 50, 60)**




**相等搜索概率下，搜索成功时：**

$$\mathbf{ASL_{succ}} = \frac{1}{n} * \sum_{i=0}^{n-1}(i+1) = \frac{n+1}{2}$$

掉进红框(红框不比较)

**相等搜索概率下，搜索失败时：**

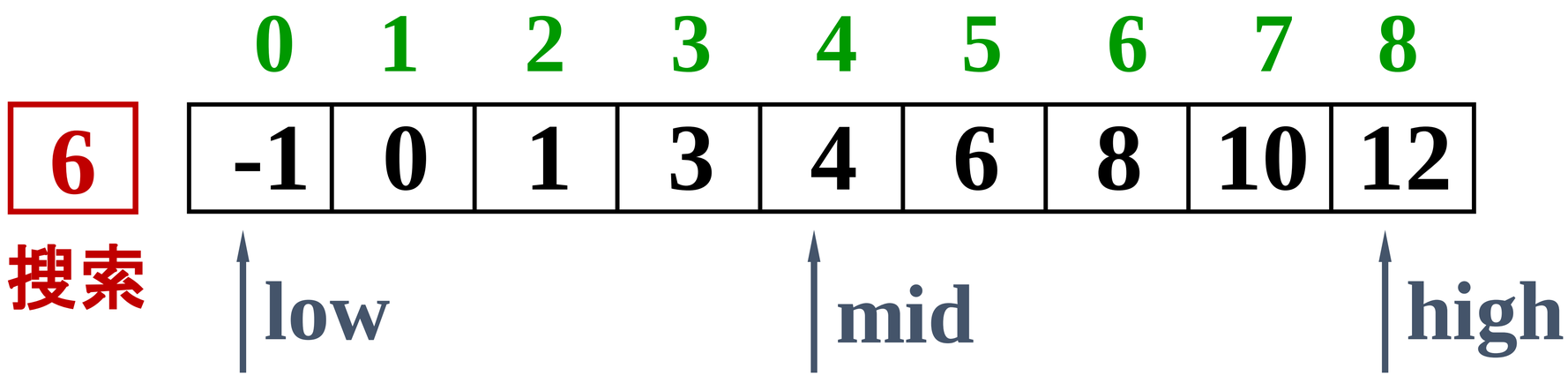
$$\mathbf{ASL_{fail}} = \sum_{i=0}^{n} q_i * li$$

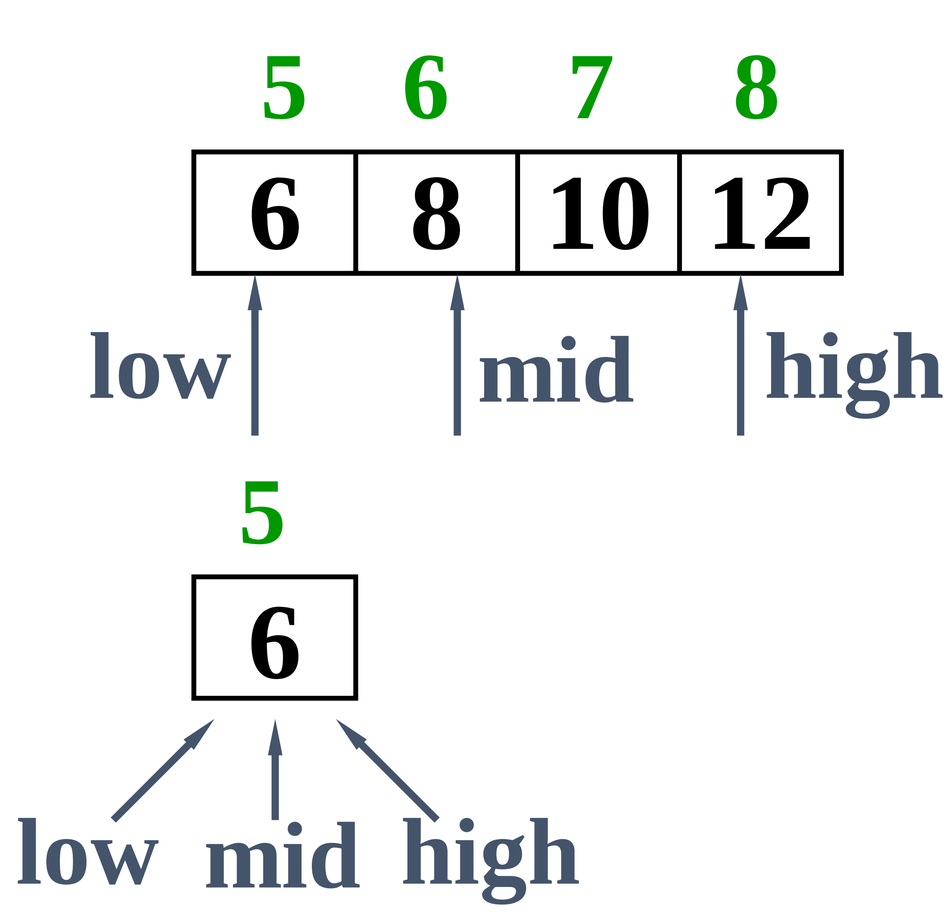
$$= \frac{1}{n+1}(1+2+\cdots+n+n) = \frac{n}{2} + \frac{n}{n+1}$$

# 有序顺序表上的折半搜索

- **折半搜索(Binary Search，又称二分搜索)：循环执行以下步骤直到找到或者确定所找数据对象不存在**
  - **与当前搜索区间的中间位置的数据对象进行比较**
  - **若该数据对象与要找的数据对象相等，则搜索成功**
  - **若该数据对象比要找的小，搜索区间缩小为当前区间的后半部分**
  - **若该数据对象比要找的大，搜索区间缩小为当前区间的前半部分**
- **每次循环，搜索区间减半，最多$\log_2 n$次循环就可以完成**

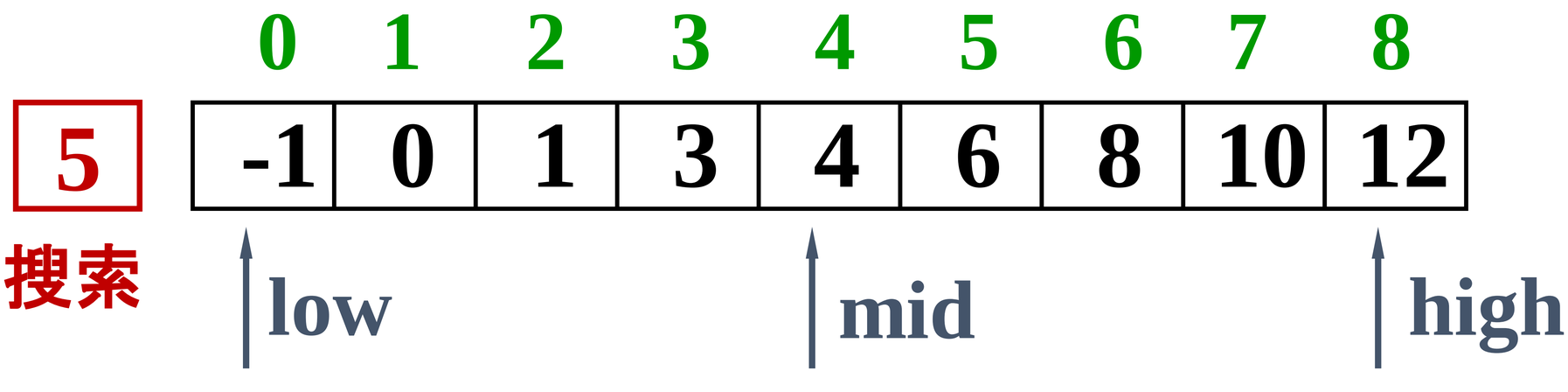
# 折半搜索成功的示例

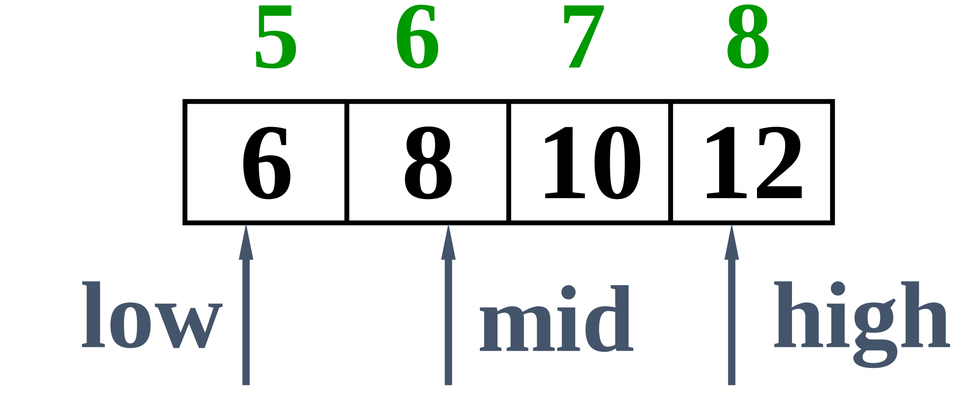

0 1 2 3 4 5 6 7 8
6
-1 0 1 3 4 6 8 10 12
搜索
low
mid
high

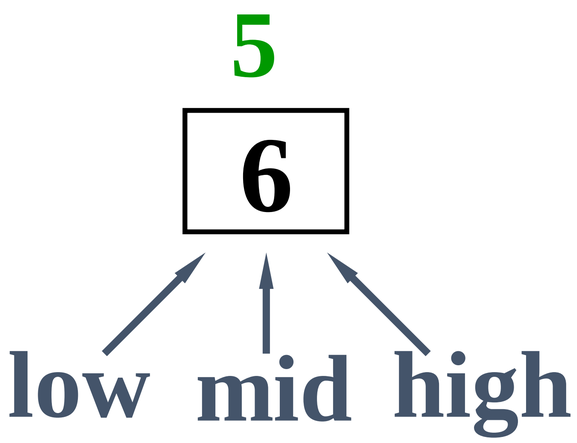

5 6 7 8
6 8 10 12
low
mid
high


5
6
low
mid
high

# 折半搜索失败的示例


0 1 2 3 4 5 6 7 8
5
-1 0 1 3 4 6 8 10 12
搜索
low
mid
high


5 6 7 8
6 8 10 12
low
mid
high


5
6
low
mid
high

# 折半搜索的递归算法

```
template <class Type> int SortedList<Type> ::
BinarySearch ( const Type & x, const int low, const int high ) const {
    int mid = −1;
    if ( low <= high ) {
        mid = ( low + high ) / 2;
        if ( Element[mid].key < x )
            mid = BinarySearch ( x, mid +1, high );
        else if ( Element[mid].key > x )
            mid = BinarySearch ( x, low, mid −1 );
    }
    return mid;
}
```

# 折半搜索的迭代算法

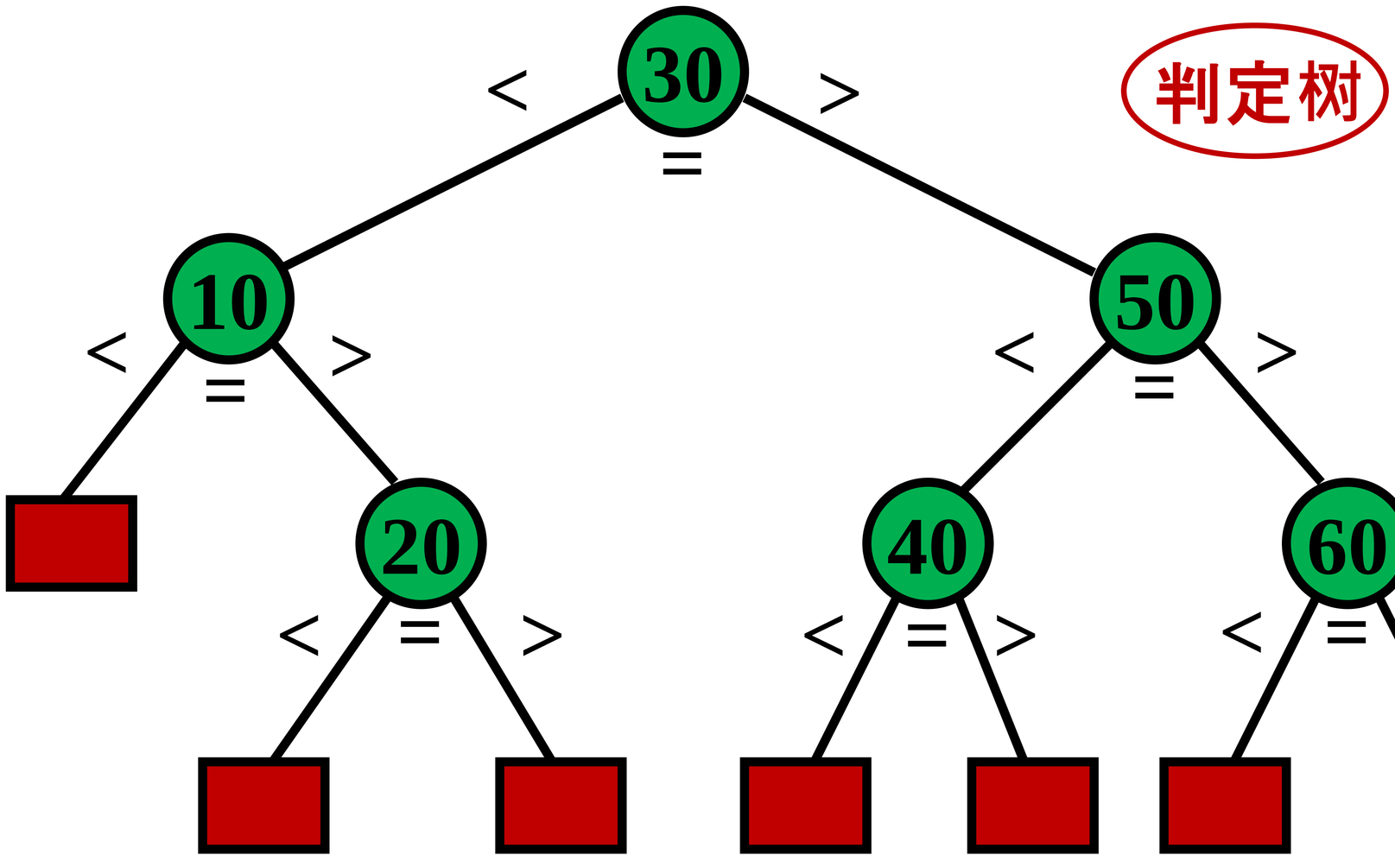
```
template <class Type> int SortedList <Type> ::
BinarySearch(const Type &x) const {
    int high = CurrentSize - 1, low = 0, mid;
    while ( low <= high ) {
        mid = (low + high)/2;
        if ( Element[mid].key < x )
            low = mid + 1;  // 右缩搜索区间
        else if ( Element[mid].key > x )
            high = mid - 1;  // 左缩搜索区间
        else return mid;  // 搜索成功
    }
    return -1;  // 搜索失败
}
```

# 折半搜索示例

- 按关键码有序的顺序表
  - 关键码依次为(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)
  - 要搜索关键码为8的数据对象
- 搜索过程
  - 首先和顺序表中间结点5比较
  - 由于8比5大，搜索区间缩小为后面一半
  - 和新搜索区间(6, 7, 8, 9, 10, 11)的中间结点8进行比较，找到
- 这个例子中，折半搜索需要2次比较就可以找到最终的数据对象，而顺序搜索则需要9次比较才能找到

# 折半搜索的平均搜索长度

**(10, 20, 30, 40, 50, 60)**



# 折半搜索的平均搜索长度

- 若设$n=2^h-1$，则描述折半搜索的判定树是高度为h的满二叉树($2^h = n+1, h = \log_2(n+1)$)
- 第1层结点有1个，搜索第1层结点要比较1次；第2层结点有2个，搜索第2层结点要比较2次；…；第i(1≤i≤h)层结点有$2^{i-1}$个，搜索第i层结点要比较i次；…
- 假定每个结点的搜索概率相等，即$p_i=1/n$，则搜索成功的平均搜索长度$ASL = \sum_{i=0}^{n-1} p_i * ci = \frac{1}{n}(1 * 2^0 + 2 * 2^1 + \cdots + \log_2(n+1) * 2^{(\log_2(n+1)-1)})$

# 有序顺序表上的插值搜索

- **插值搜索(Interpolating Search)：若知道数据的分布情况，可以根据搜索数据的关键码预估其在顺序表中的位置**

```
template < class Type> int SortedList <Type> ::
InterpolatingSearch(const Type &x) const {
    int low = 0, high = CurrentSize - 1, mid;
    while( low <= high ) {
        // 先根据插值公式计算中值，假设数据是均匀分布          //中值占比
        mid = (x - Element[low].key) / (Element[high].key - Element[low].key);
        mid = mid * (high - low) + low;
        if( Element[mid].key < x)  low = mid + 1;
        else if( Element[mid].key > x)  high = mid - 1;
        else  return mid;
    }
    return -1;
}
```

# 插值搜索示例

- 按关键码有序的顺序表
  - 关键码依次为(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)
  - 要搜索关键码为8的数据对象
- 搜索过程
  - 由于8在0～11的区间中的位置是72.7%，首先和顺序表中位于72.7%位置的数据对象进行比较
  - 在这个位置上的数据对象就是8，1次比较就找到了

$\frac{8}{11}$

# 有序顺序表上的斐波那契搜索

- 插值搜索的前提是数据在顺序表中分布是接近均匀分布的，但很多时候数据的分布是不均匀的，而且其分布也是事先未知的。为了追求更好的平均性能，可采用**斐波那契搜索**，其基础是斐波那契数
  - **F(n) = n，　n = 0, 1**
  - **F(n) = F(n-1) + F(n-2)，　n >= 2**

# 斐波那契搜索算法

- 假设初始时下标从0开始的顺序表a中元素个数为m
  - 先找到一个最小的n，使得F(n)-1 >= m
  - 如果F(n)-1 > m，则用a[m-1]填充a[m]到a[F(n)-2]
  - 在搜索前，a的长度恰好为F(n)-1，设low = 0，high = m-1，k = n
- 接下来只要low <= high，就反复做以下计算
  - mid = low + F(k-1) - 1
  - 如果a[mid]符合要求，则搜索成功，返回
  - 如果a[mid]大于要找的元素，则high = mid-1
    - 新的high-low区间长度=F(k-1)-1，可以继续循环
  - 否则，low = mid + 1
    - 新的high-low区间长度=F(k)-1-F(k-1) = F(k-2)-1，可以继续循环

# 斐波那契搜索算法

```
template < class Type> int SortedList <Type> ::
FibSearch(const Type &x) const {
    int k, low = 0, high = CurrentSize - 1, mid;
    // 一系列准备工作，构造Fib数组F，确定k的初值，…(略)
    while( low <= high ) {
        mid = low + F[k-1] -1;
        if( Element[mid].key == x) {
            if( mid < CurrentSize ) return mid;
            else return CurrentSize - 1; //填充
        } else if( Element[mid].key < x) { low = mid + 1; k -= 2; }//长度变为F(k-2)-1
        else { high = mid - 1; k--; } // 长度变为 F(k-1) -1
    }
    return -1;
}
```



# 斐波那契搜索的特点

- 优点
  - **搜索空间的缩减速度近乎黄金分割的比例，对大部分情况的数据分布具有比较好的性能**
    - 平均性能比折半搜索好
    - 时间复杂度仍然是$O(\log_2 n)$
  - **计算mid只要做加减法，不要做除法**
    - 似乎意义不大，因为整数除以2**可以用移位**实现
- **缺点**
  - **需要预先构造Fib数组F，需要更多的时间和空间消耗**
    - 如果经常做这种搜索，可以预先构造好数组F，**就不要每次搜索都构造**

# 练习1

- 对有序链表能进行顺序和折半搜索吗？为什么？
- **能顺序搜索，因为顺序搜索对元素存储没有限制**
- **不能折半搜索，因为折半搜索要求采用顺序存储**

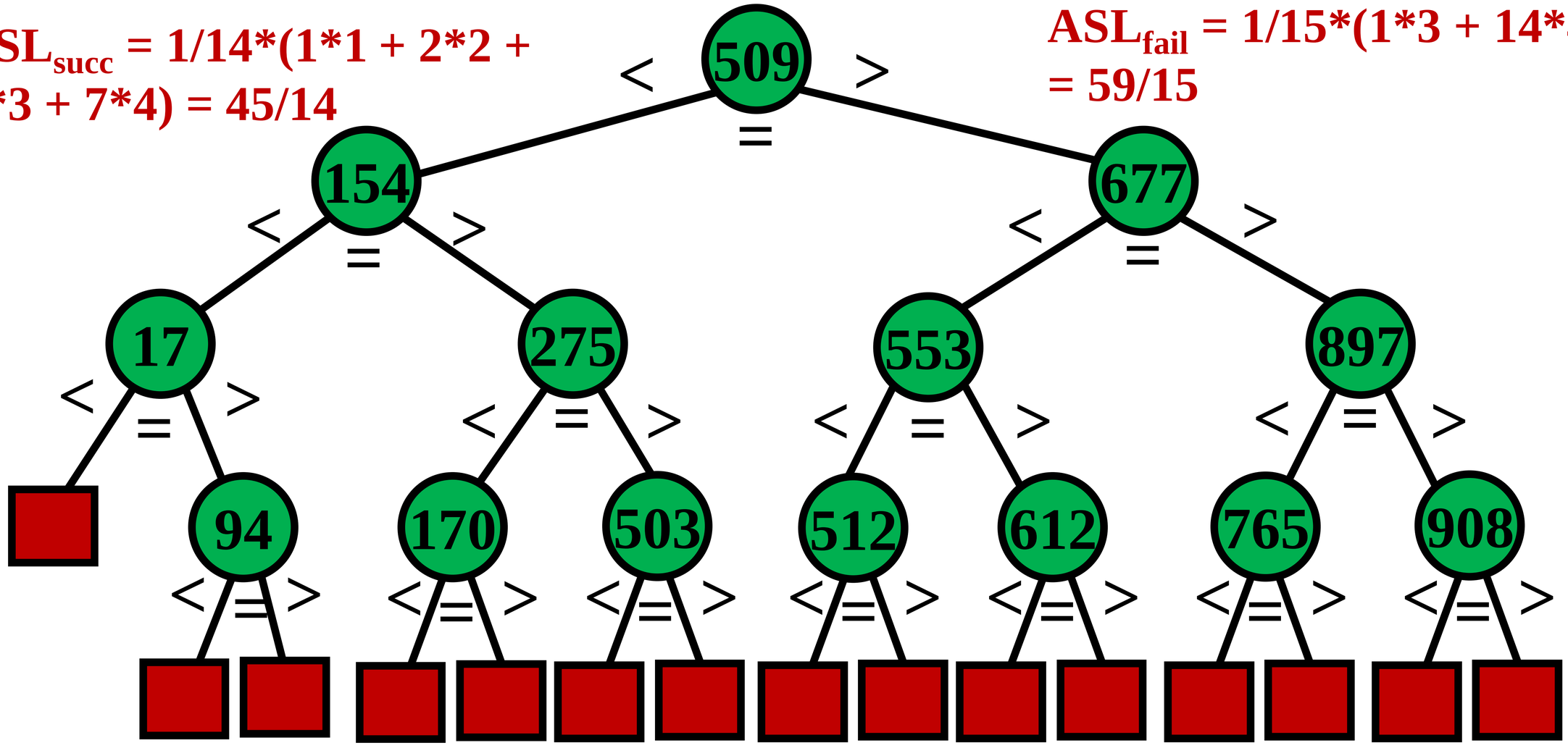
- **已知有序顺序表(13, 18, 24, 35, 47, 50, 62, 83, 90, 115, 134)，当折半搜索值为18的元素时，需要几次数据比较？**
- **4次，搜索序列为50, 24, 13, 18**

# 练习1

- 已知有序顺序表(17, 94, 154, 170, 275, 503, 509, 512, 553, 612, 677, 765, 897, 908)，画出对其进行折半搜索时的判定树，计算等概率下搜索成功/失败的平均搜索长度。

$ASL_{succ}$ = 1/14*(1*1 + 2*2 + 4*3 + 7*4) = 45/14

$ASL_{fail}$ = 1/15*(1*3 + 14*4) = 59/15



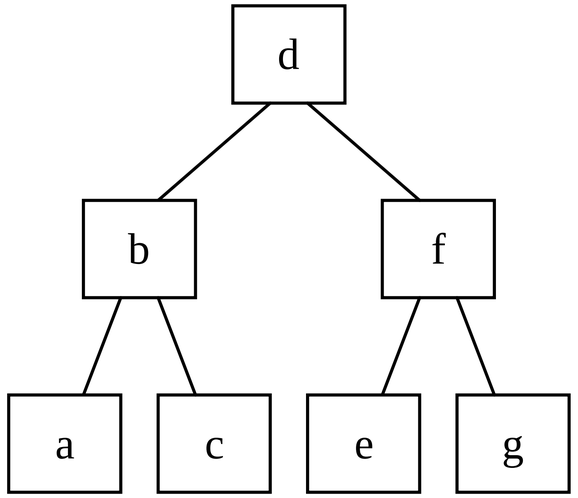
# 07 搜索与索引

- 搜索与索引
- 基于顺序表的搜索
- 二叉搜索树

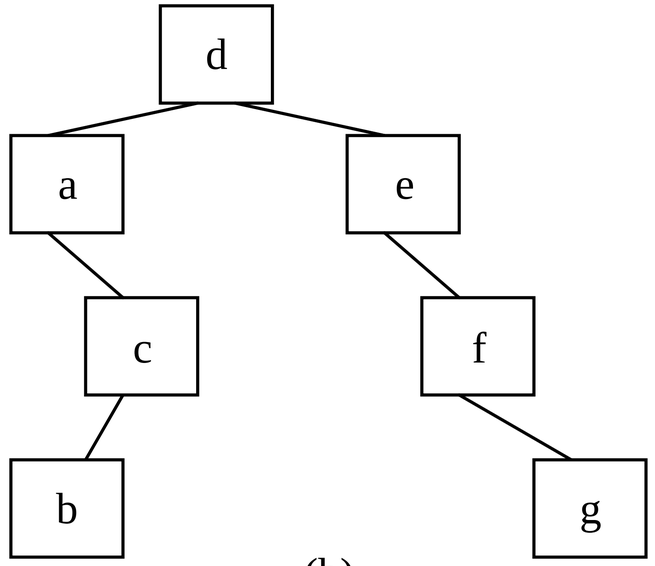
# 二叉搜索树

- **二叉搜索树(Binary Search Tree)T是一棵二叉树，它或为空，或满足下面4个条件**
  - 每个结点有三个属性：键值、左孩子指针、右孩子指针
  - 如果T的根结点的左子树非空，那么左子树中的所有结点的键值都小于T的根结点的键值
  - 如果T的根结点的右子树非空，那么右子树中的所有结点的键值都大于等于T的根结点的键值
  - T的根结点的左右子树也都是二叉搜索树
- **若中序遍历二叉搜索树T的结点，可得到一个排好序的结点序列，故也称T为二叉排序树**

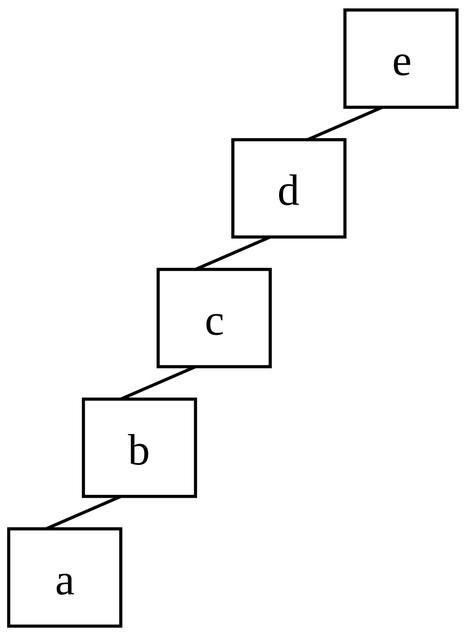
# 二叉搜索树示例



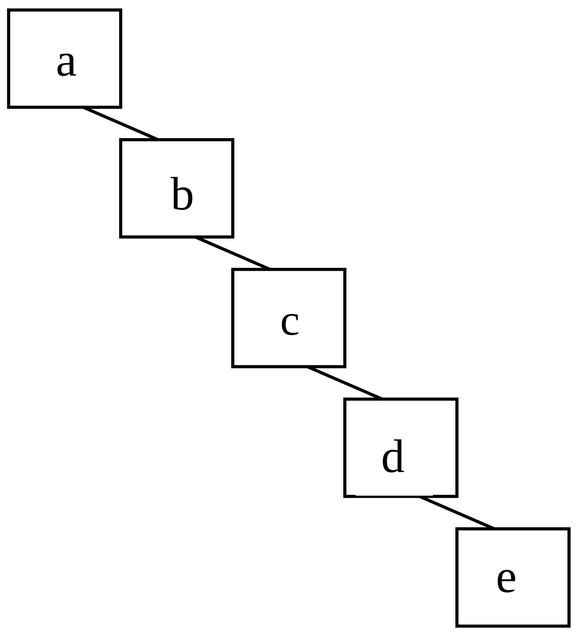

(a)




(b)




(c)




(d)

# 二叉搜索树的类定义

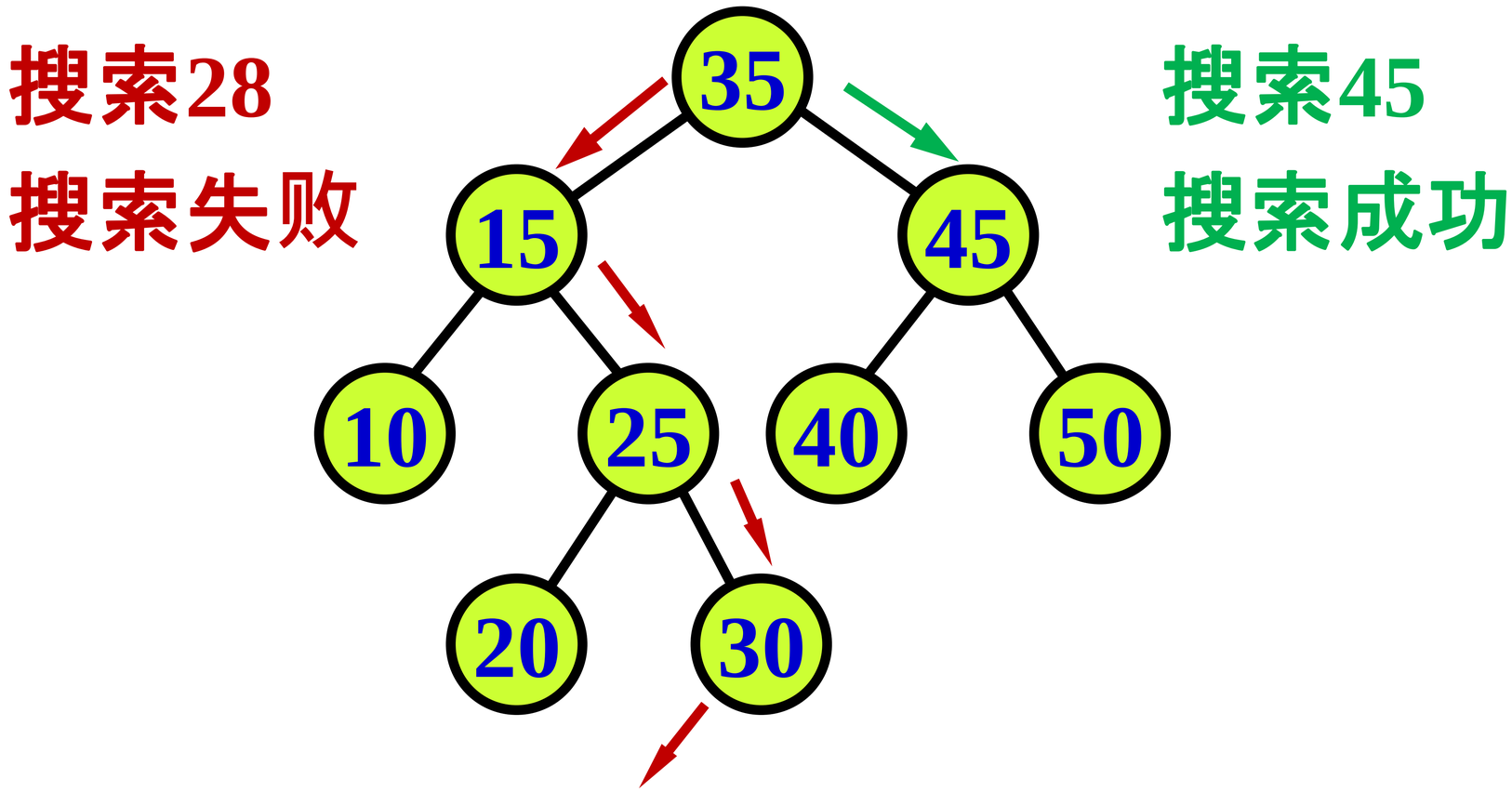
```
template <class Type> class BST;
template <class Type> class BstNode {
    friend class BST <Type>;
    protected:
        Type data;
        BstNode <Type> *leftChild, *rightChild;
    public:
        BstNode( ) : leftChild(NULL), rightChild(NULL) { }  // 构造函数
        BstNode(const Type d, BstNode <Type> *L=NULL,
            BstNode <Type> *R=NULL)
            : data(d), leftChild(L), rightChild(R) { }
        void SetData(Type d) { data = d; }
        Type GetData( ) { return data; }
        …
};
```

# 二叉搜索树的类定义

```
template <class Type> class BST {
    private:
        BstNode <Type> *root;  // 根指针
        Type RefValue;  // 数据输入停止的标志
        BstNode <Type> * Search(Type x, BstNode <Type> *ptr);  // 搜索
        bool Insert(Type x, BstNode <Type> *&ptr);  // 插入
        bool Remove(Type x, BstNode <Type> *&ptr);  // 删除
        …
    public:
        BST( ) : root(NULL) { }  // 构造函数
        BST(Type value);  // 构造函数
        bool Search(Type x) { return Search(x, root) != NULL; }
        bool Insert(Type x) { return Insert(x, root); }
        bool Remove(Type x) { return Remove(x, root); }
        …
};
```

# 二叉搜索树上的搜索

- 二叉搜索树t中搜索包含给定关键码x的结点算法
  - 如果t为空，那么搜索失败，算法结束
  - 如果t->data == x，那么搜索成功，算法结束
  - 如果t->data > x，那么继续在其左子树中搜索
  - 否则，即t->data < x，那么继续在其右子树中搜索



# 二叉搜索树上搜索的递归算法

```
template <class Type> BstNode <Type> * BST<Type> ::
Search(Type x, BstNode <Type> *ptr) const {
    if (ptr == NULL)  return NULL;  // 搜索失败
    else if (x < ptr->data)  // 在左子树搜索
        return Search(x, ptr->leftChild);
    else if (x > ptr->data)  // 在右子树搜索
        return Search(x, ptr->rightChild);
    else return ptr;  // 相等，搜索成功
}
```

# 二叉搜索树上搜索的迭代算法

```
template <class Type> BstNode <Type> * BST <Type> ::
Search(Type x, BstNode <Type> *ptr) const {
    if (ptr != NULL) {
        BstNode <Type> *temp = ptr;  // 从根搜索
        while (temp != NULL) {
            if (temp->data == x)  return temp;  // 搜索成功
            if (temp->data < x) temp = temp->rightChild;  // 右子树
            else temp = temp->leftChild;  // 左子树
        }
    }
    return NULL;  // 搜索失败
}
```

# 平均搜索长度

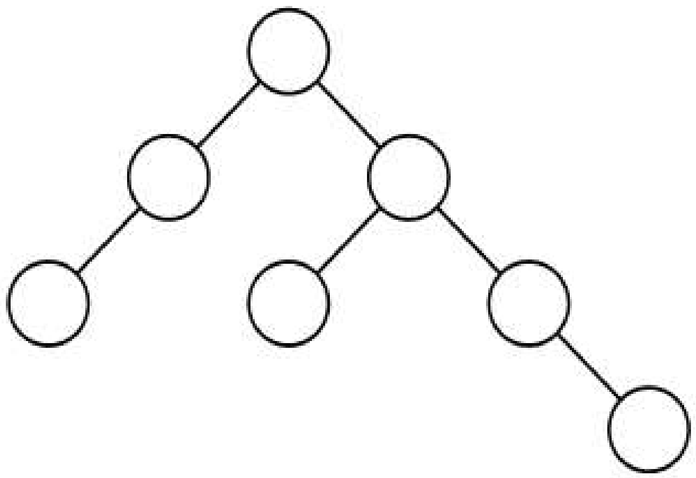
- 对于成功的搜索，从n个结点的二叉搜索树T中找出结点x所需的比较次数是从根结点到结点x的边的数量λ(x)+1
  - 最大搜索长度 = max{λ(x)+1 | T中的所有结点x}
  - 平均搜索长度 = Σp(x)(λ(x)+1)，p(x)是结点x被搜索的概率。若所有结点被搜索的概率相等，那么对任一结点x都有p(x)=1/n，则平均搜索长度 = 1/n * Σ(λ(x)+1)
- 显然，T的高度越小，平均搜索长度越小
  - 当T退化成单链表，平均搜索长度最大

# 最优二叉搜索树

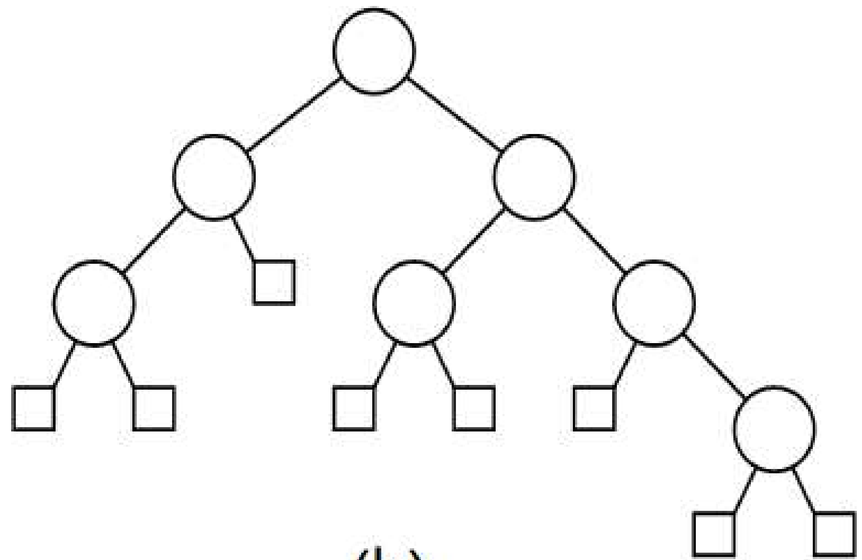
- **最优二叉搜索树**：对一个特定的数据集，搜索的复杂性最小的二叉搜索树
  - 不仅要考虑搜索成功的情况，还要考虑搜索失败的情况
  - 不仅考虑搜索概率相同的情况，还要考虑搜索概率不相同的情况

# 扩充二叉树

- 在二叉树中，在结点的每个空指针上都挂上一个附加结点，称为**外部结点**(用方框表示)，而称原来树中的结点为**内部结点**(用圆表示)，扩充后的树称为**扩充二叉树**



(a)



(b)

# 路径长度

总指针：2n　　边数：n-1　　空指针数：n+1

- 具有n个结点的二叉树，经过扩充后产生n+1个外部结点
  - 对于扩充的二叉搜索树，失败的搜索总是终止于一个外部结点，外部结点代表失败的搜索，故外部结点也称为失败结点
- 称由根结点到所有内部结点的路径长度(即经过的边的数量)总和为二叉树的内部路径长度$I_n$
  - 上页扩充二叉树的内部路径长度$I_n$= 0+1+1+2+2+2+3 = 11
- 称由根结点到所有外部结点的路径长度总和为二叉树的外部路径长度$E_n$
  - 上页扩充二叉树的外部路径长度$E_n$ = 3+3+2+3+3+3+4+4 = 25

# 内外部路径长度的关系

- **一棵有n个内部结点的扩充二叉树的内部路径和外部路径长度之间的关系可用公式$E_n = I_n + 2n$表示(归纳法证明)**



  - **当n=1时，由于只有一个内部结点，所以$I_1 = 0$，$E_1 = 2$，满足**
  - **假设所有有n个内部结点的扩充二叉树都满足$E_n = I_n + 2n$**
  - **对于有n个内部结点的任一扩充二叉树，把树中一个路径长为L的外部结点变成一个内部结点，在此结点下附加两个路径长为L+1的外部结点，变成了具有n+1个内部结点的扩充二叉树，且满足**
    - $I_{n+1} = I_n + L$，$E_{n+1} = E_n - L + 2(L+1)$
    - 得到$I_n = I_{n+1} - L$，$E_{n+1} = E_n + L + 2$
    - 得到$E_{n+1} = I_n + 2n + L + 2 = I_{n+1} - L + 2n + L + 2 = I_{n+1} + 2n + 2 = I_{n+1} + 2(n+1)$



# 最大/小内部路径长度

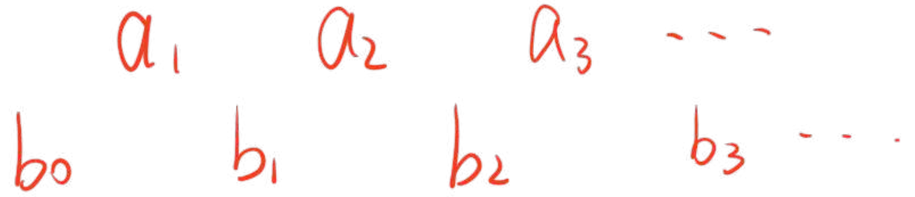
- 对于有n个结点的二叉树，若具有最大(或最小)内部路径长度，也一定具有最大(或最小)外部路径长度；反之亦然

- 当二叉树退化成单链表时，其内部路径长度最大
  - $\mathbf{I_n} = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$

- 当内部结点都尽量靠近根结点时，其内部路径长度最小
  - $\mathbf{I_n} = 1 * 0 + 2 * 1 + 4 * 2 + \cdots = \sum_{i=1}^{n} \lfloor log_2 i \rfloor$

# 最优二叉搜索树

- 如果二叉搜索树由内部结点序列$a_1<a_2<\ldots<a_n$构成，n+1个外部结点为$b_0, b_1, \ldots, b_n$，当搜索值为x的结点时
  - 如果$a_i<x<a_{i+1}$，那么在$b_i$上终止搜索
  - 如果$x<a_1$，那么在$b_0$终止搜索
  - 如果$x>a_n$，那么在$b_n$终止搜索




- 设$p_i(1\le i\le n)$是$a_i$的搜索概率，而$q_j(0\le j\le n)$是$b_j$的搜索概率
  - 成功搜索的平均搜索长度为$\sum_{i=1}^{n} p_i(\lambda(a_i)+1)$
  - 失败搜索的平均搜索长度为$\sum_{i=0}^{n} q_i*\lambda(b_i)$
  - 搜索的总的平均搜索长度为$\sum_{i=1}^{n} p_i(\lambda(a_i)+1)+\sum_{i=0}^{n} q_i*\lambda(b_i)$，且$\sum_{i=1}^{n} p_i+\sum_{i=0}^{n} q_i=1$

- 由结点$a_1, a_2, \ldots, a_n$构成的所有可能的二叉搜索树中，使总的平均搜索长度取最小值的，就是**最优二叉搜索树**

# 搜索概率相等的情况

- 如果成功搜索和失败搜索都具有相同的搜索概率，即$p_i = q_i = 1/(2n+1)$时，总的平均搜索长度为
  - $\mathbf{ASL} = \frac{1}{2n+1}\left(\sum_{i=1}^{n}(\lambda(a_i)+1) + \sum_{i=0}^{n}\lambda(b_i)\right)$
  - $\mathbf{ASL} = \frac{1}{2n+1}(I_n + n + En) = \frac{1}{2n+1}(2In + 3n)$
  - 只要$I_n$最小，ASL就达到最小

- 显然，满的二叉搜索树能使$I_n$最小
  - $I_n = \sum_{i=1}^{n}\lfloor log_2 i \rfloor \leq nlog_2 n = O(nlog_2 n)$
  - $\mathbf{ASL} = \frac{1}{2n+1}(2In + 3n) = O(log_2 n)$

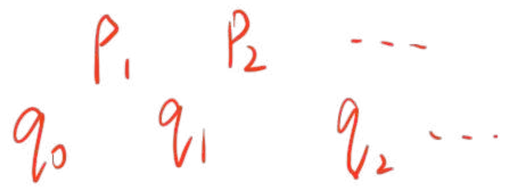
$\log_2 i \leq \log_2 n$

# 搜索概率不相等的情况

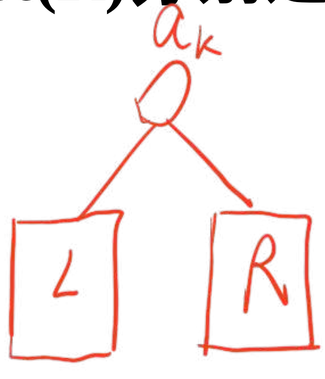
- 给定n个内部结点$a_1, a_2, \ldots, a_n$的搜索概率为$p_1, p_2, \ldots, p_n$，n+1个外部结点的搜索概率为$q_0, q_1, \ldots, q_n$，如何构造搜索概率不相等的情况下的最优二叉搜索树？
  - 办法一：构造出所有可能的二叉搜索树，并选ASL最小的树，但随着n的增加，不同二叉搜索树的个数迅速增加，计算复杂性高
  - 办法二：根据最优二叉搜索树的性质，有一种高效的构造算法

# 构造最优二叉搜索树

- 用$T_{ij}$表示由$a_{i+1}, \ldots, a_j (i<j)$组成的一棵最优二叉搜索树，且约定：当$0 \le i=j \le n$时，$T_{ij}$为空树；当$i>j$时，$T_{ij}$无定义
  - 用$c_{ij}$表示$T_{ij}$的代价，$c_{ii} = 0$
  - 用$r_{ij}$表示$T_{ij}$的根，$r_{ii} = 0$
  - 用$w_{ij}=q_i + \sum_{k=i+1}^{j}(p_k + q_k)$表示$T_{ij}$的权，$w_{ii} = q_i$




- 若$T_{ij}$的根$r_{ij}=k (i<k \le j)$，L为根$a_k$的左子树，由$a_{i+1}, \ldots, a_{k-1}$组成，R是根$a_k$的右子树，由$a_{k+1}, \ldots, a_j$组成
  - $c_{ij} = p_k + cost(L) + w_{ik-1}$ + $cost(R) + w_{kj}$ $= w_{ij} + cost(L) + cost(R)$
  - 其中cost(L)和cost(R)分别是左子树L和右子树R的代价



# 构造最优二叉搜索树

- 最优二叉搜索树具有这样的性质：一棵最优二叉搜索树的所有子树都是最优二叉搜索树 局部最优性
  - 故有$c_{ij} = w_{ij} + cost(L) + cost(R) = w_{ij} + c_{ik-1} + c_{kj}$
- 由于$T_{ij}$是最优二叉搜索树，故满足
  - $c_{ij} = w_{ij} + c_{ik-1} + c_{kj} = \min_{i<t\le j}\{w_{ij} + c_{it-1} + c_{tj}\}$
  - 即有$c_{ij} = w_{ij} + c_{ik-1} + c_{kj} = w_{ij} + \min_{i<t\le j}\{c_{it-1} + c_{tj}\}$
  - 而$w_{ij} = w_{ij-1} + p_j + q_j$
- 基于上述关系，可以从已形成的较小的最优二叉搜索树出发，系统地构造越来越大的最优二叉搜索树

w: weight

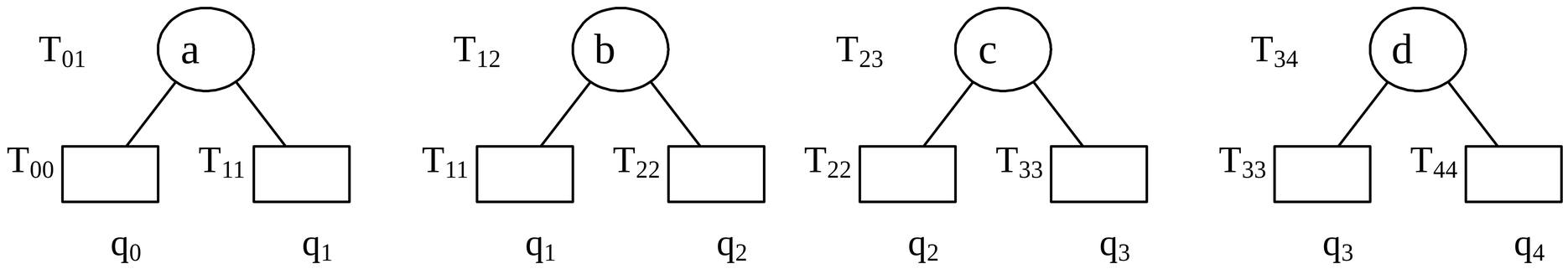
# 构造过程示例

- 设n=4，且$(a_1, a_2, a_3, a_4) = (a, b, c, d)$，取$(p_1, p_2, p_3, p_4) = (3, 3, 1, 1)$，$(q_0, q_1, q_2, q_3, q_4) = (2, 3, 1, 1, 1)$
- 初始时，取$w_{ii}=q_i$，$c_{ii}=0$和$r_{ii}=0$ $(0<=i<=4)$
- 在下面的叙述中，为书写方便，把$T_{ij}$的根结点的左子树和右子树分别记为$T^l_{ij}$和$T^r_{ij}$

# 1个内部结点的情况

- 当j-i=1时
  - 因为 $w_{01} = w_{00} + p_1 + q_1 = 8$ $c_{01} = w_{01} + c_{00} + c_{11} = 8$ 所以 $r_{01} = 1$ $\mathrm{T}_{01}^{l} = \mathrm{T}_{00}$ $\mathrm{T}_{01}^{r} = \mathrm{T}_{11}$
  - 因为 $w_{12} = w_{11} + p_2 + q_2 = 7$ $c_{12} = w_{12} + c_{11} + c_{22} = 7$ 所以 $r_{12} = 2$ $\mathrm{T}_{12}^{l} = \mathrm{T}_{11}$ $\mathrm{T}_{12}^{r} = \mathrm{T}_{22}$
  - 因为 $w_{23} = w_{22} + p_3 + q_3 = 3$ $c_{23} = w_{23} + c_{22} + c_{33} = 3$ 所以 $r_{23} = 3$ $\mathrm{T}_{23}^{l} = \mathrm{T}_{22}$ $\mathrm{T}_{23}^{r} = \mathrm{T}_{33}$
  - 因为 $w_{34} = w_{33} + p_4 + q_4 = 3$ $c_{34} = w_{34} + c_{33} + c_{44} = 3$ 所以 $r_{34} = 4$ $\mathrm{T}_{23}^{l} = \mathrm{T}_{22}$ $\mathrm{T}_{23}^{r} = \mathrm{T}_{33}$

# 1个内部结点的情况


T01
a
T00
T11
q0
q1
T12
b
T11
T22
q1
q2
T23
c
T22
T33
q2
q3
T34
d
T33
T44
q3
q4

# 2个内部结点的情况

- **当j-i=2时**

$$w_{02} = w_{01} + p_2 + q_2 = 12$$

- **因为** $c_{02} = w_{02} + \min\{ c_{00} + c_{12},\ \ c_{01} + c_{22}\}$ **所以** $r_{02} = 1 \quad \mathrm{T}_{02}^{l} = \mathrm{T}_{00} \quad \mathrm{T}_{02}^{r} = \mathrm{T}_{12}$

$$c_{00} + c_{12} < c_{01} + c_{22}$$

$$w_{13} = w_{12} + p_3 + q_3 = 9$$

- **因为** $c_{13} = w_{13} + \min\{ c_{11} + c_{23},\ \ c_{12} + c_{33}\}$ **所以** $r_{13} = 2 \quad \mathrm{T}_{13}^{l} = \mathrm{T}_{11} \quad \mathrm{T}_{13}^{r} = \mathrm{T}_{23}$

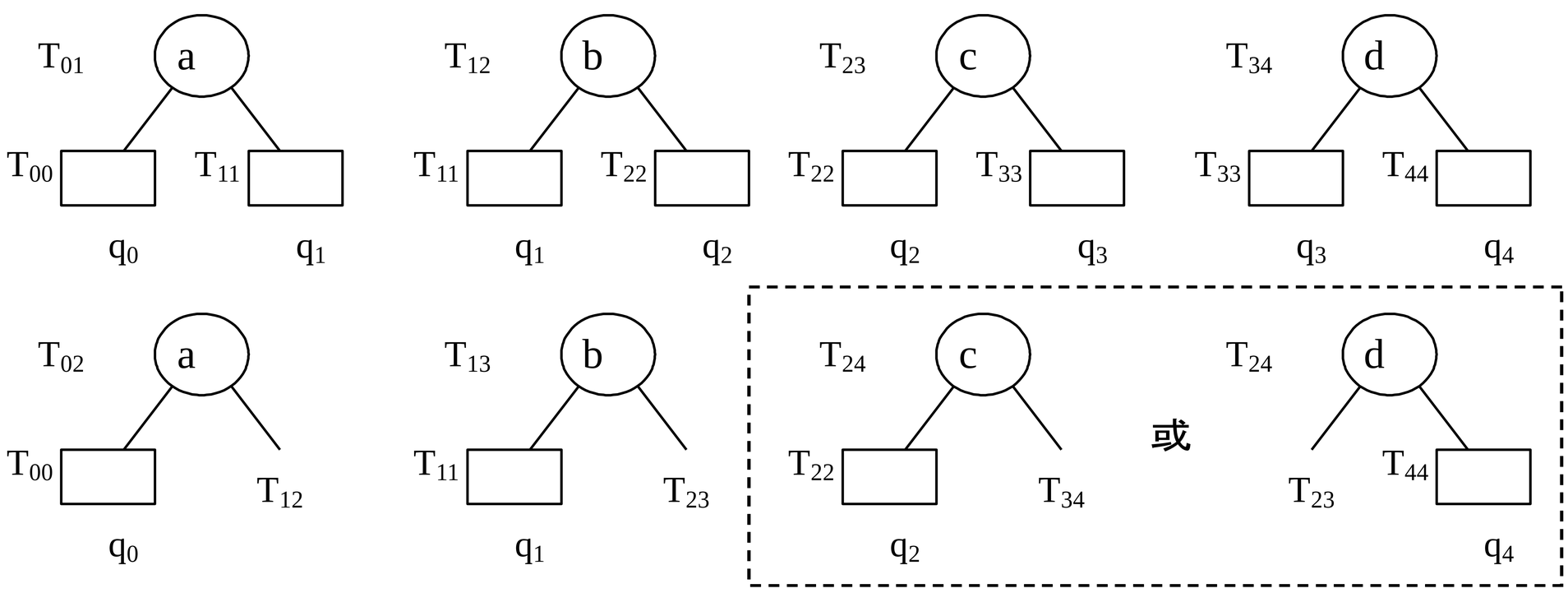
$$c_{11} + c_{23} < c_{12} + c_{33}$$

$$w_{24} = w_{23} + p_4 + q_4 = 5$$

- **因为** $c_{24} = w_{24} + \min\{ c_{22} + c_{34},\ \ c_{23} + c_{44}\}$ **所以** $r_{24} = 3 \quad \mathrm{T}_{24}^{l} = \mathrm{T}_{22} \quad \mathrm{T}_{24}^{r} = \mathrm{T}_{34}$ 或 $r_{24} = 4 \quad \mathrm{T}_{24}^{l} = \mathrm{T}_{23} \quad \mathrm{T}_{24}^{r} = \mathrm{T}_{44}$

$$c_{22} + c_{34} = c_{23} + c_{44}$$

# 2个内部结点的情况


T01
a
T00
T11
q0
q1
T12
b
T11
T22
q1
q2
T23
c
T22
T33
q2
q3
T34
d
T33
T44
q3
q4
T02
a
T00
T12
q0
T13
b
T11
T23
q1
T24
c
T22
T34
q2
或
T24
d
T23
T44
q4

# 3个内部结点的情况

- **当j-i=3时**

$$w_{03} = w_{02} + p_3 + q_3 = 14$$

- **因为** $c_{03} = w_{03} + \min\{ c_{00} + c_{13},\ \ c_{01} + c_{23},\ \ c_{02} + c_{33}\} = 25$

$$c_{01} + c_{23} < c_{00} + c_{13},\ \ c_{01} + c_{23} < c_{02} + c_{33}$$

- **所以** $r_{03} = 2$ $\qquad \mathrm{T}_{03}^{l} = \mathrm{T}_{01}$ $\qquad \mathrm{T}_{03}^{r} = \mathrm{T}_{23}$

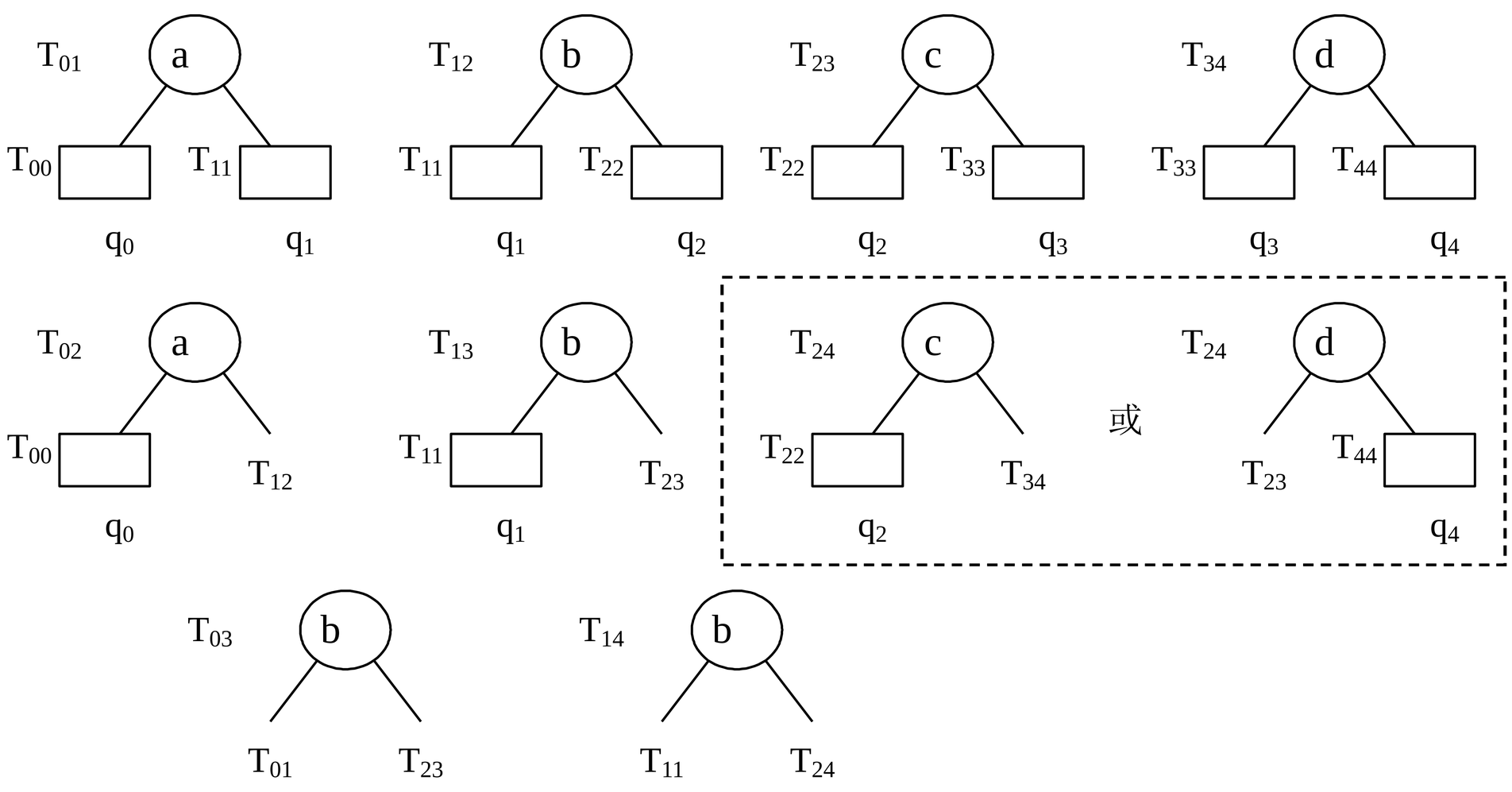
$$w_{14} = w_{13} + p_4 + q_4 = 11$$

- **因为** $c_{14} = w_{14} + \min\{ c_{11} + c_{24},\ \ c_{12} + c_{34},\ \ c_{13} + c_{44}\} = 19$

$$c_{11} + c_{24} < c_{12} + c_{34},\ \ c_{11} + c_{24} < c_{13} + c_{44}$$

- **所以** $r_{14} = 2$ $\qquad \mathrm{T}_{14}^{l} = \mathrm{T}_{11}$ $\qquad \mathrm{T}_{14}^{r} = \mathrm{T}_{24}$

# 3个内部结点的情况


T01
a
T00
T11
q0
q1
T12
b
T11
T22
q1
q2
T23
c
T22
T33
q2
q3
T34
d
T33
T44
q3
q4
T02
a
T00
T12
q0
T13
b
T11
T23
q1
T24
c
T22
T34
q2
或
T24
d
T23
T44
q4
T03
b
T01
T23
T14
b
T11
T24

# 构造出整棵最优二叉搜索树

- **当j-i=4时**

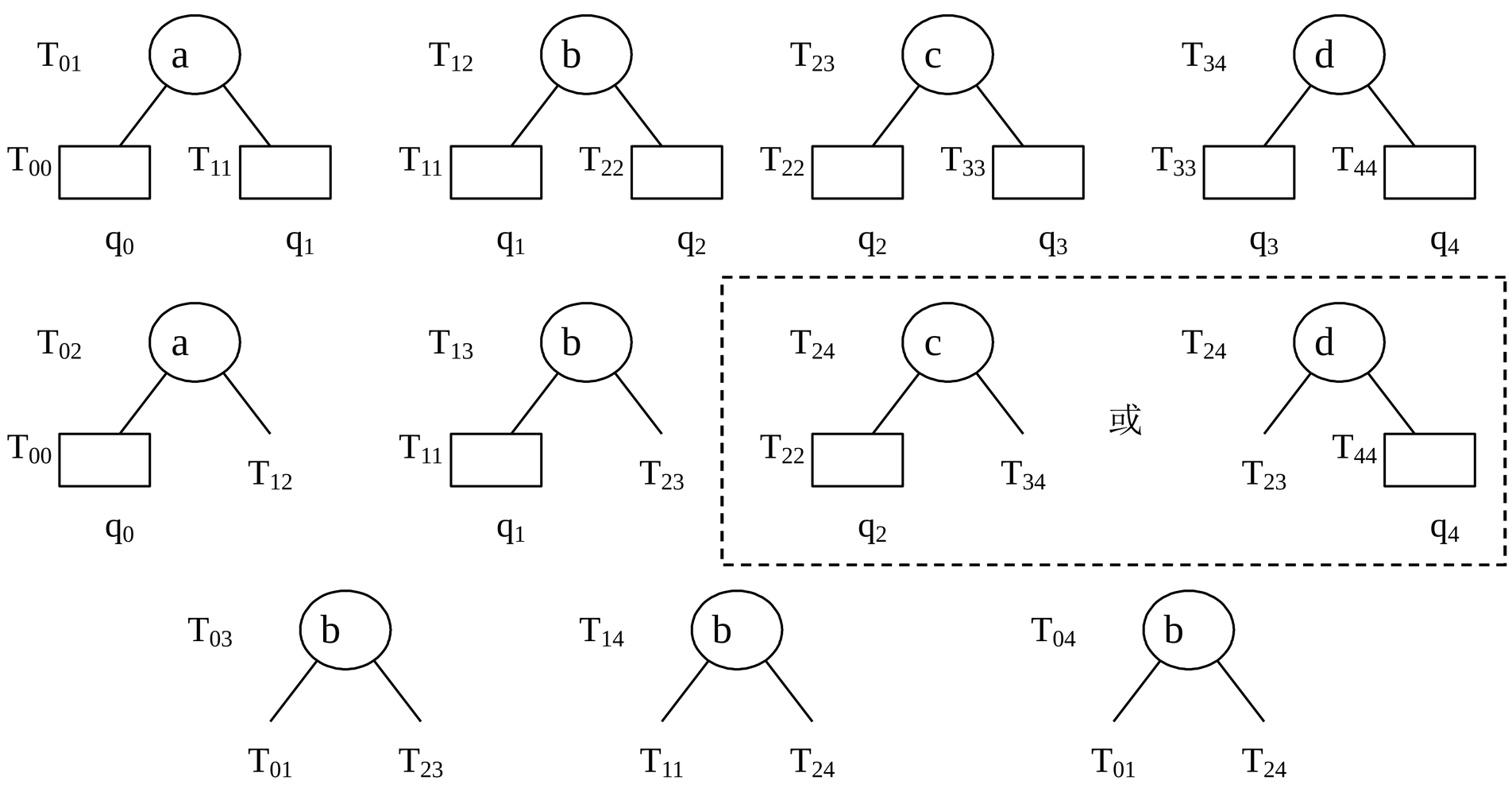
$$w_{04} = w_{03} + p_4 + q_4 = 16$$

- **因为** $c_{04} = w_{04} + \min\{ c_{00} + c_{14},\ \ c_{01} + c_{24},\ \ c_{02} + c_{34},\ \ c_{03} + c_{44}\} = 32$

$$c_{01} + c_{24} < \ c_{00} + c_{14},\ \ c_{02} + c_{34},\ \ c_{03} + c_{44}$$

- **所以** $r_{04} = 2 \qquad \mathrm{T}_{04}^{l} = \mathrm{T}_{01} \qquad \mathrm{T}_{04}^{r} = \mathrm{T}_{24}$

# 构造出整棵最优二叉搜索树



# 算法基本思路

- 初始化
  - W[i][i], C[i][i], W[i][i+1], C[i][i+1], R[i][i+1]
- 循环(内部结点数(当前规模)从2增到n，步长为1)
  - 循环(依次对当前规模的每棵子树)
    - 循环(依次让本子树内每个内部结点为根)
      - {根据之前算好的较小规模子树的代价，推算当前子树的代价，从中寻找最小代价子树，保存，用于后续推算更大规模子树的代价}
- 最后得到最优二叉搜索树

# 算法性能分析

- 主要时间消耗在计算各种$c_{ij}$
  - 在计算$c_{ij}$时，按j-i=1, 2, …, n的次序进行
  - 当j-i=1时，计算$c_{01}, c_{12}, \ldots, c_{n-1\ n}$
  - 当j-i=2时，计算$c_{02}, c_{13}, \ldots, c_{n-2\ n}$
  - ……
  - 当j-i=m时，计算n-m+1个$c_{ij}$
  - 计算$c_{ij}$时需要从m个数中找出其最小者
  - 故所需的时间为$(n-m+1)m = nm-m^2+m$
  - m是从1取到n，故总时间为

$$\sum_{m=1}^{n}(nm-m^2+m)=\frac{1}{6}(n^3+3n^2+2n)=\frac{1}{6}n(n+1)(n+2)=O(n^3)$$

$$1^2+2^2+\cdots+n^2=\frac{n(n+1)(2n+1)}{6}$$

# 动态二叉搜索树

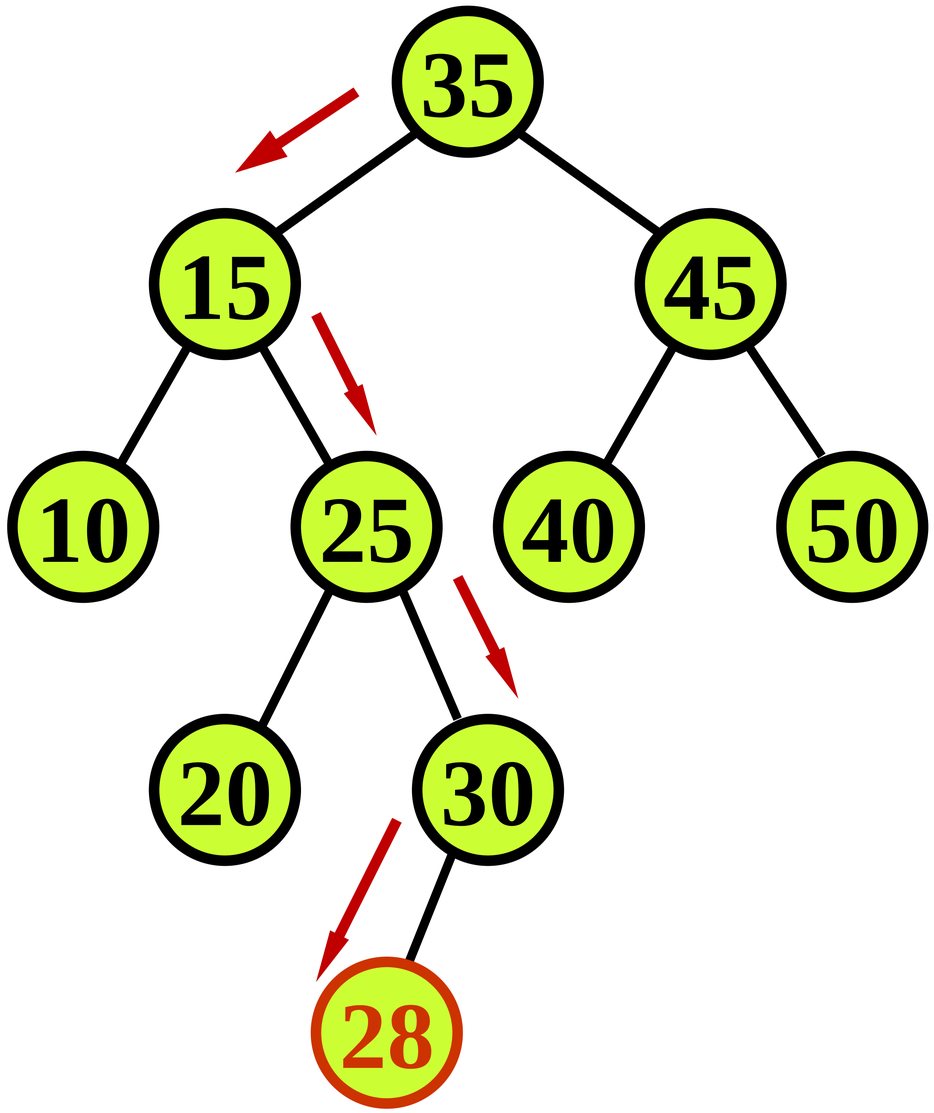
- 前面介绍了静态二叉搜索树的基本结构，在静态二叉搜索树基础上可以通过插入和删除操作实现对数据的动态操作，故称为动态二叉搜索树
  - 非平衡二叉搜索树
  - 高度平衡二叉搜索树

# 二叉搜索树的插入操作

- **使用搜索算法**检查待插入的元素是否已经在树中
  - **如果搜索成功**，则说明待插入的元素已经在树中，不再插入
  - **如果搜索失败**，则把新元素插入到搜索停止的地方

```
template <class Type> bool BST <Type> ::
Insert(Type x, BstNode <Type> *&ptr) {
    if (ptr == NULL) {  // 空二叉搜索树
        ptr = new BstNode <Type> (x);  // 创建新根结点
        return true;
    } else if (x < ptr->data)  // 在左子树插入
        Insert(x, ptr->leftChild);
    else if (x > ptr->data)  // 在右子树插入
        Insert(x, ptr->rightChild);
    else return false;  //插入失败，待插入元素已在树中
}
```

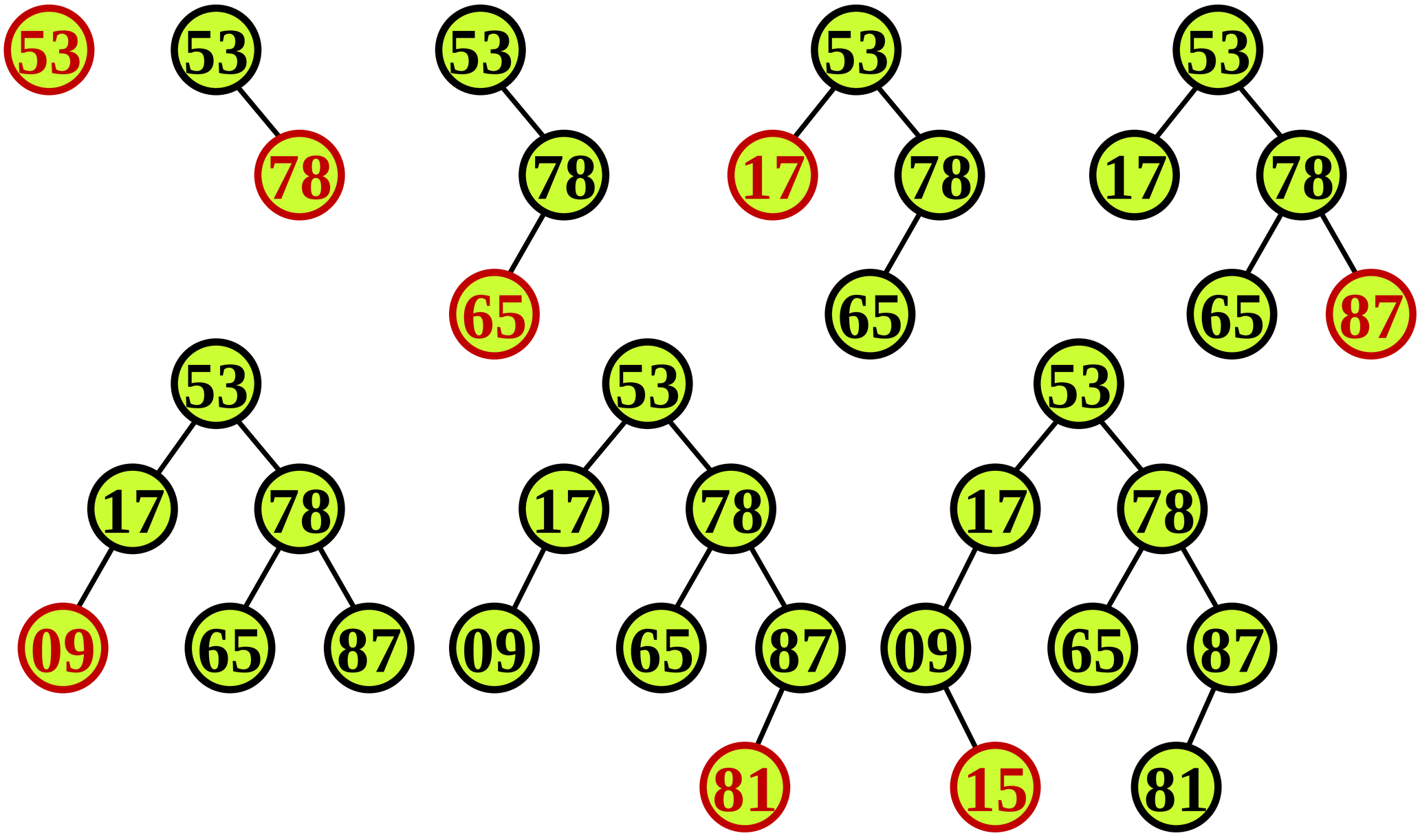
# 插入操作示例




插入新结点28

# 输入数据建立二叉搜索树

**输入数据 {53, 78, 65, 17, 87, 09, 81, 15}**



# 输入数据建立二叉搜索树

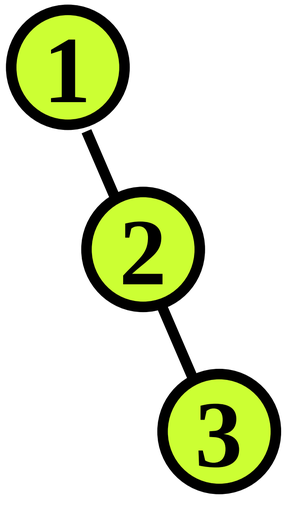
```
template <class Type> BST <Type> :: BST(Type value) {
    Type x;
    root = NULL;
    RefValue = value;  // RefValue是输入停止的标志
    cin>>x;
    while (x != RefValue) {
        Insert(x, root);
        cin>>x;
    }
}
```

# 输入顺序不同，将影响到建立的二叉搜索树的形态和搜索性能

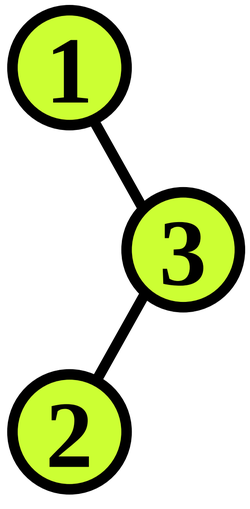
**若输入序列选得不好，会建立一棵单支树，使得二叉搜索树的高度达到最大，影响搜索性能**

**{2, 1, 3}**

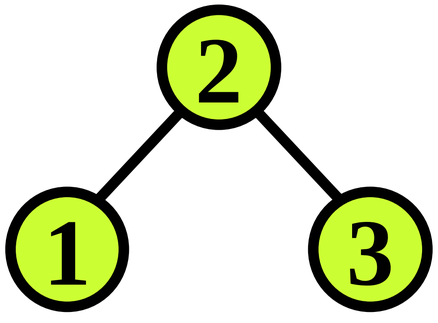
**{1, 2, 3}**



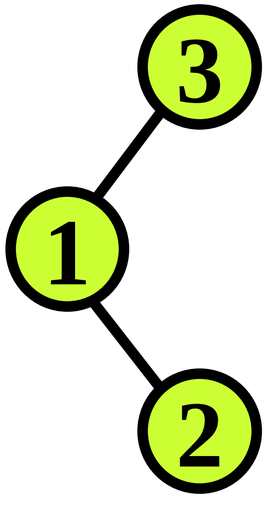

**{1, 3, 2}**



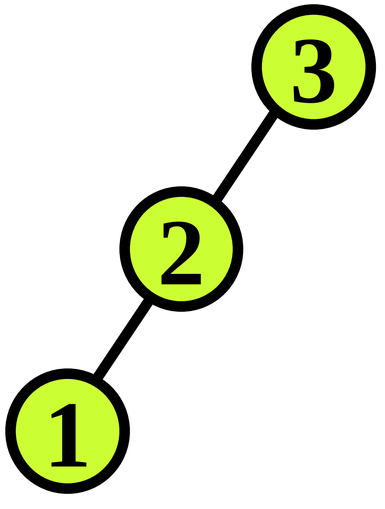

**{2, 3, 1}**




**{3, 1, 2}**



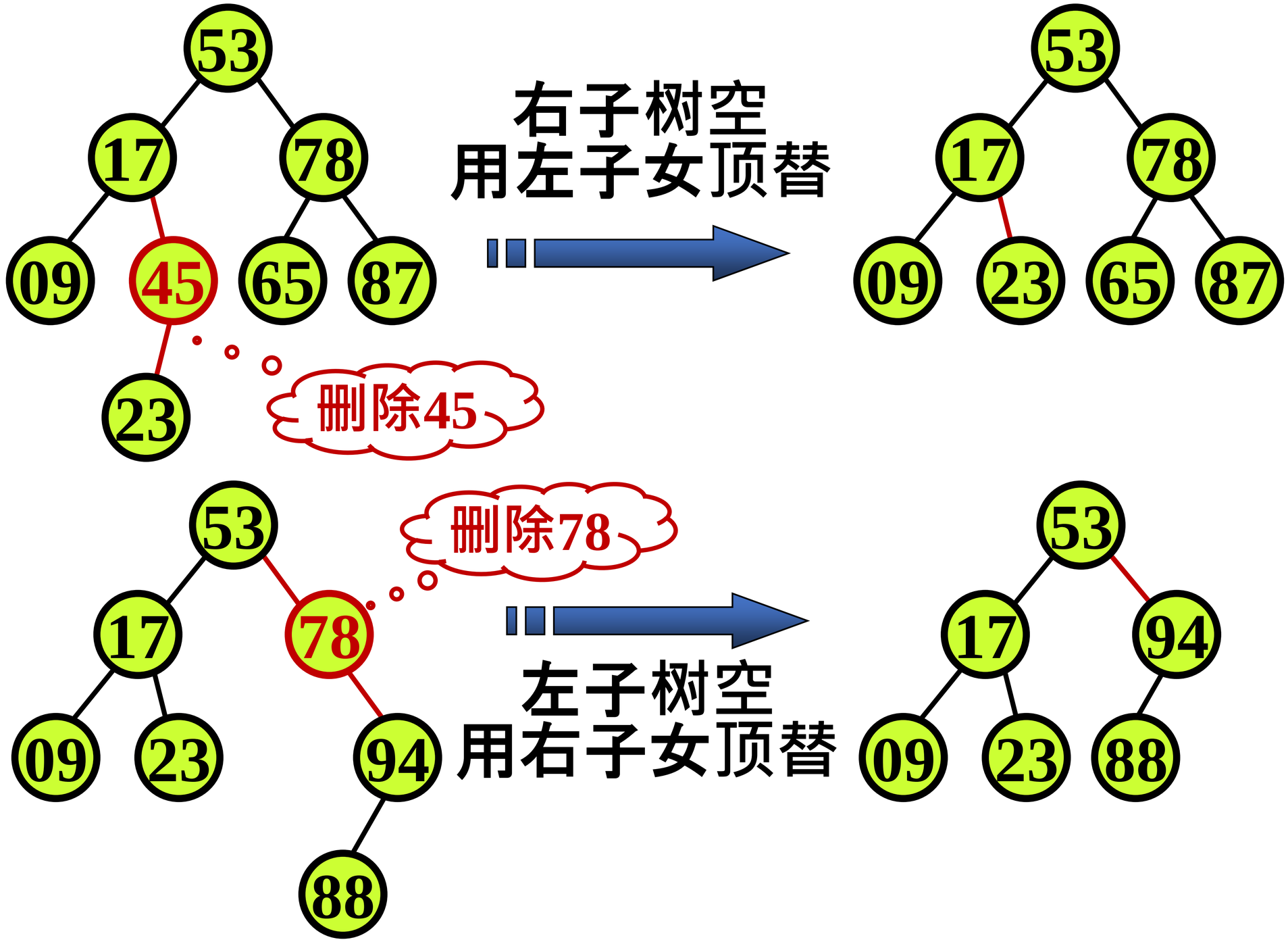

**{3, 2, 1}**



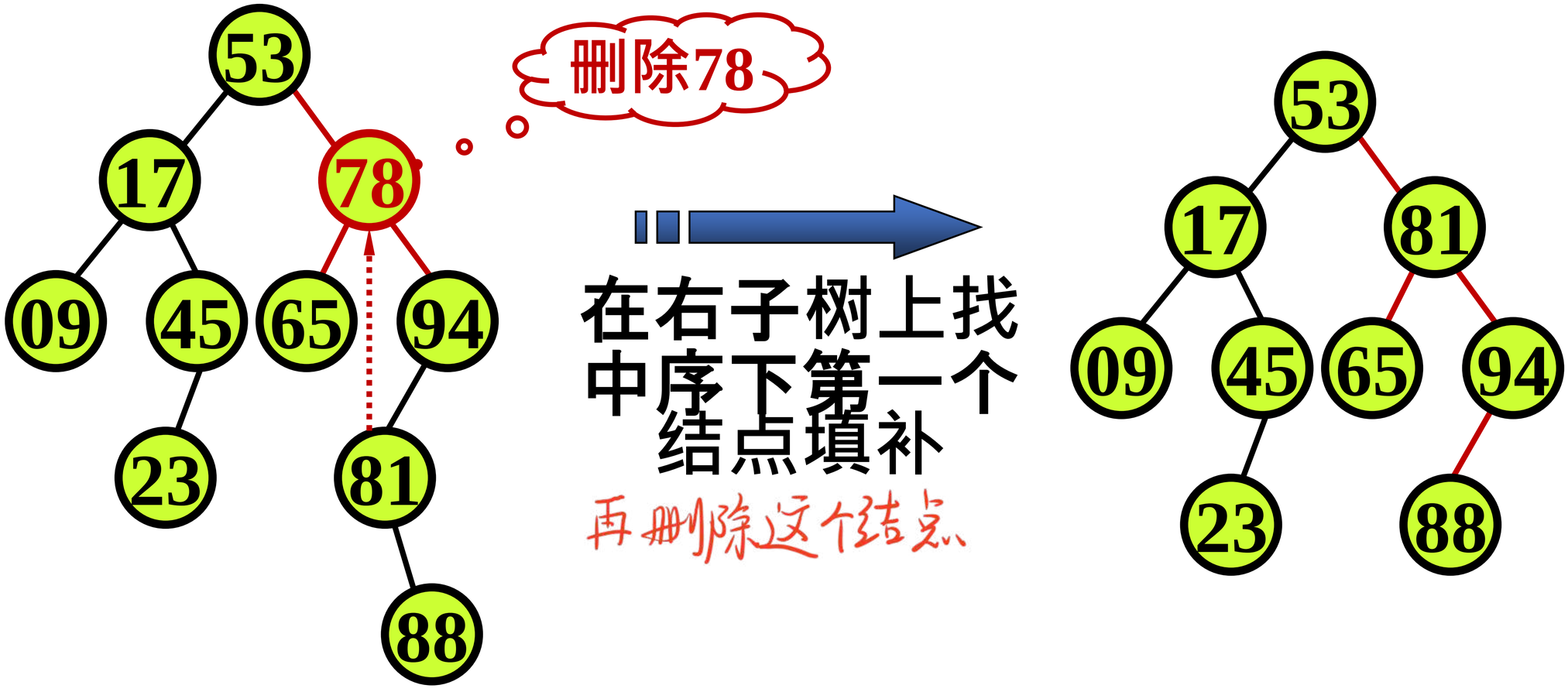
# 二叉搜索树的删除操作

- 在删除一个结点时，必须将因删除结点而断开的二叉链表重新链接起来，同时确保二叉搜索树的性质不会失去
- 为保证在删除后二叉搜索树的搜索性能不至于降低，还需要防止重新链接后树的高度增加
  - 删除叶结点：将其双亲结点指向它的指针清零，再释放它
  - 被删结点右子树为空：拿它的左子女顶替它的位置，再释放它
  - 被删结点左子树为空：拿它的右子女顶替它的位置，再释放它
  - 被删结点左、右子树都不为空：在它的右子树中寻找中序下的第一个结点(关键码最小)，填补被删结点，再处理这个结点的删除

# 删除操作示例


53
17
78
09
45
65
87
23
删除45
右子树空
用左子女顶替
53
17
78
09
23
65
87
53
17
78
09
23
94
88
删除78
左子树空
用右子女顶替
53
17
94
09
23
88

# 删除操作示例


53
17
78
09
45
65
94
23
81
88
删除78
在右子树上找
中序下第一个
结点填补
再删除这个结点
53
17
81
09
45
65
94
23
88

# 二叉搜索树的删除操作

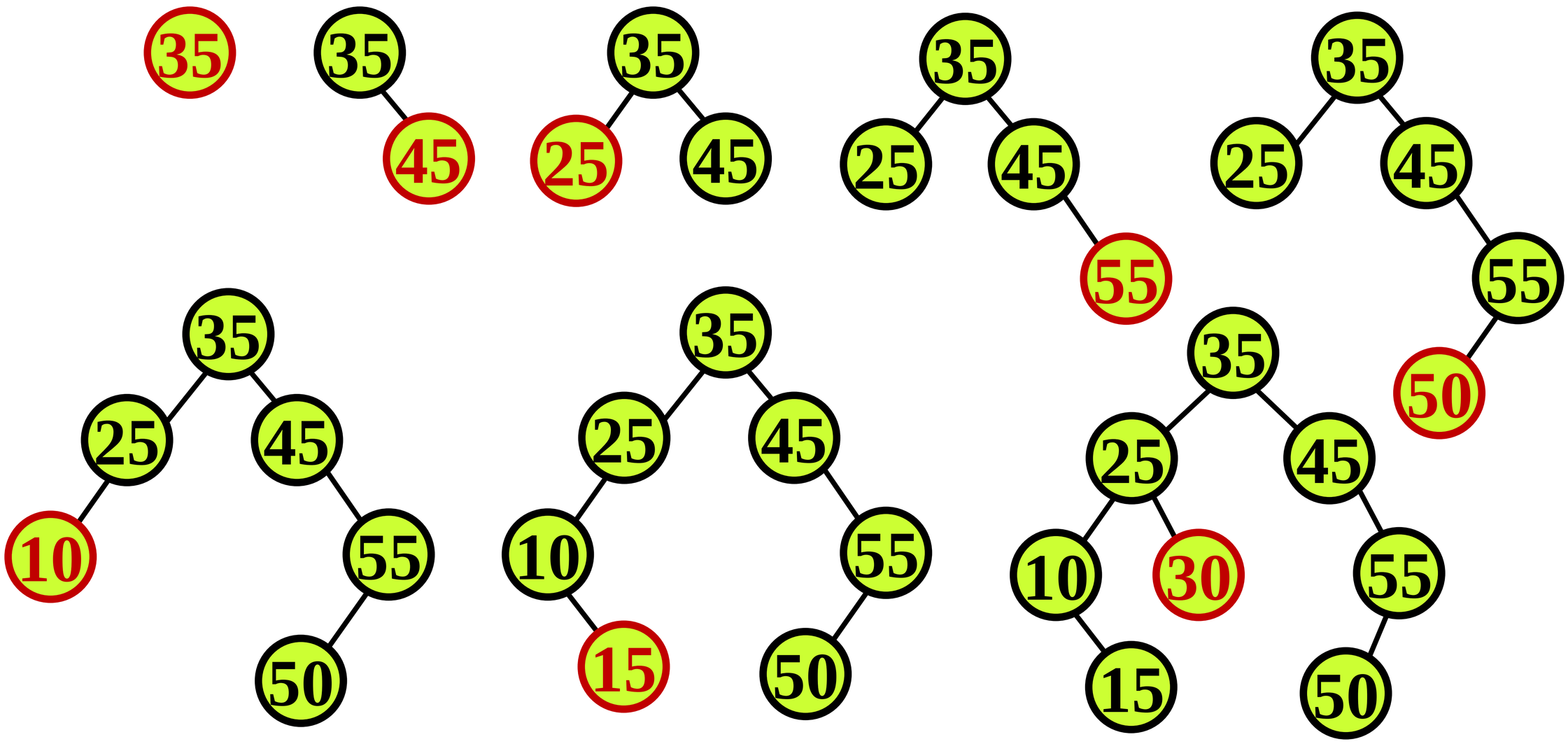
```
template <class Type> bool BST <Type> ::
Remove(Type x, BstNode <Type> *&ptr) {
    BstNode <Type> *temp;
    if (ptr != NULL)
        if (x < ptr->data)  Remove(x, ptr->leftChild);  // 在左子树中删除
        else if (x > ptr->data)  Remove(x, ptr->rightChild);  // 在右子树中删除
        else if (ptr->leftChild != NULL && ptr->rightChild != NULL) {
            temp = Min(ptr->rightChild);  // 找ptr右子树中序下第一个结点
            ptr->data = temp->data;  // 填补上
            Remove(temp->data, ptr->rightChild);  //在ptr的右子树中删除temp结点
        } else {  // ptr结点只有一个或零个子女
            temp = ptr;
            if (ptr->leftChild == NULL) ptr = ptr->rightChild;  // 只有右子女
            else if (ptr->rightChild == NULL) ptr = ptr->leftChild;  // 只有左子女
            delete temp;  return true;
        }
    return false;
}
```

# 非平衡二叉搜索树的性能分析

- 二叉搜索树的搜索、插入和删除的性能与树的高度相关，故在非平衡二叉搜索树中操作的复杂性在$\log_2 n$到n之间
  - 当非平衡二叉搜索树的高度为n时，则各操作性能较差
  - 造成这一现象的原因在于非平衡二叉搜索树过于不平衡，为了解决这个问题，需要一个平衡性较好的数据结构

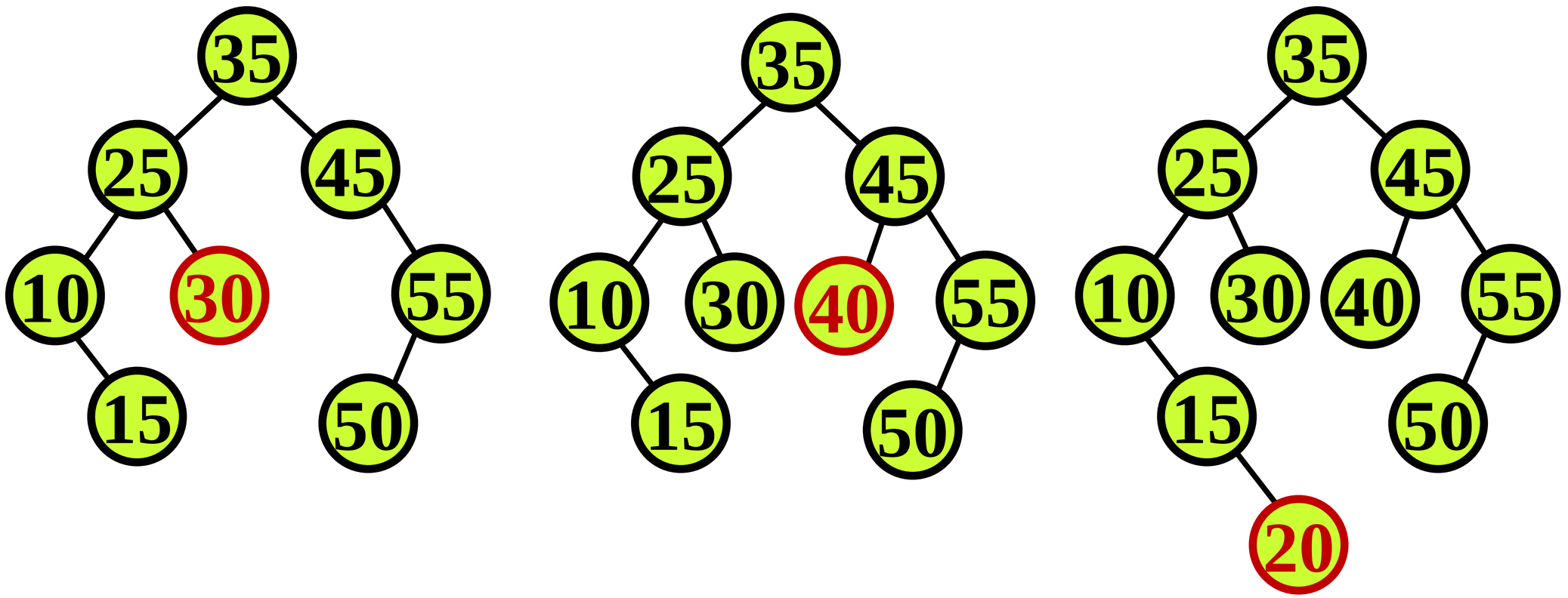
# 练习2

- **利用逐个数据插入的方式建立输入序列为35, 45, 25, 55, 50, 10, 15, 30, 40, 20所对应的二叉搜索树，搜索元素20需要进行多少元素之间的比较？**



# 练习2

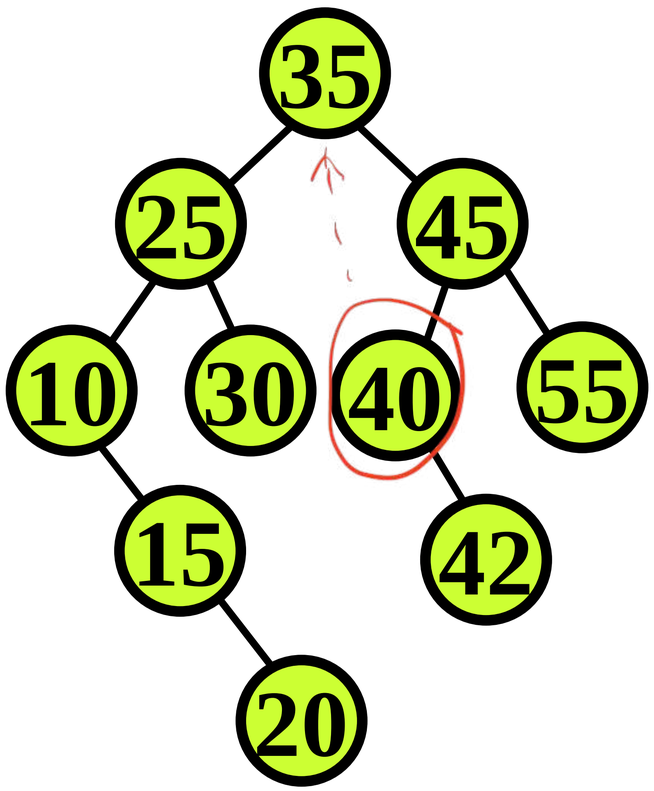
- **利用逐个数据插入的方式建立输入序列为35, 45, 25, 55, 50, 10, 15, 30, 40, 20所对应的二叉搜索树，搜索元素20需要进行多少元素之间的比较？**

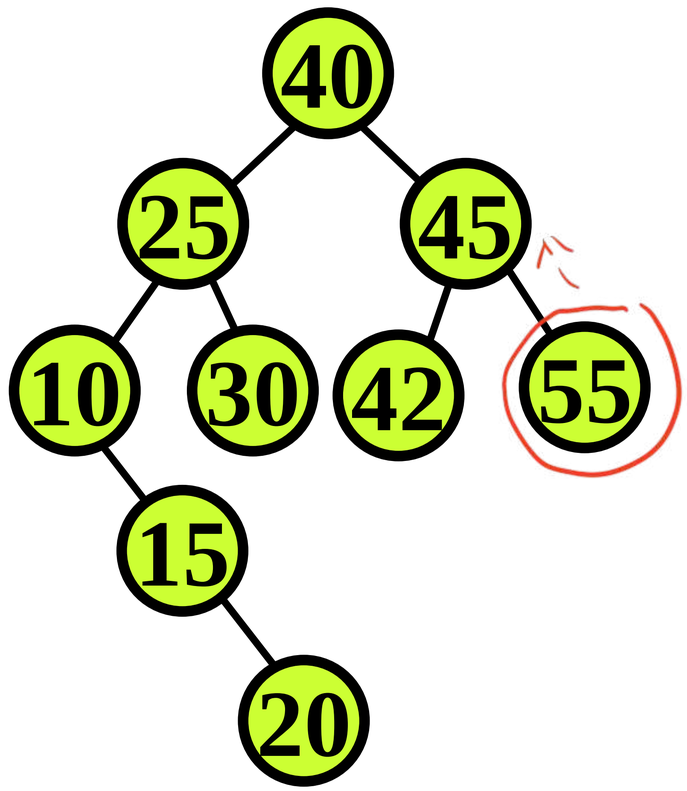



**需要5次比较**（最后一次也要比较）

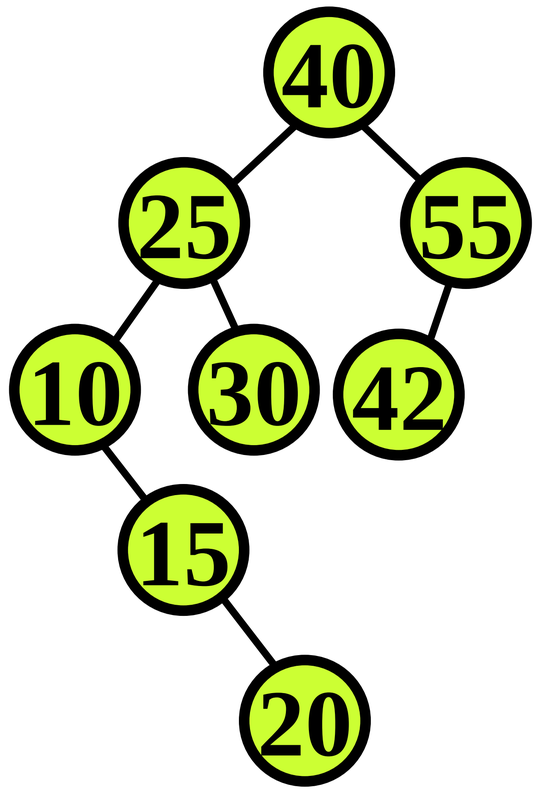
# 练习2

- 画出以此删除结点35和45后的二叉搜索树




删除35




删除45

# 本章小结

- 搜索与索引
- 基于顺序表的搜索
- 二叉搜索树

# 作业

- 设二叉树用二叉链表表示，树中结点的关键码互不相同，试编写算法判别给定的二叉树是否为二叉搜索树
- 设二叉搜索树用二叉链表表示，树中结点的关键码为整数，试编写算法将关键码小于给定值x的结点全部删除
- 在二叉搜索树上删除一个有两个子女的结点时，可以采用以下方法：用左子树上具有最大关键码的结点或者右子树上具有最小关键码的结点顶替，再递归地删除适当的结点，试编写算法实现这个删除方法